

Capítulo 11 — Context Engineering para Desarrollo de Software

Sección 01: Introducción al Context Engineering para desarrollo de software

Este capítulo no es un tutorial de herramientas de IA para programadores. Es un estudio de cómo diseñar el contexto correcto para que la IA asista efectivamente en cada fase del ciclo de vida del software. La distinción es fundamental y conviene establecerla desde el inicio.

Los tutoriales de herramientas explican cómo usar GitHub Copilot para completar código, cómo invocar Claude desde un IDE o cómo configurar un agente en Cursor. Ese conocimiento tiene valor, pero caduca rápido: las herramientas cambian, las interfaces evolucionan y los modelos se actualizan. Lo que no caduca es el principio que subyace a todas esas herramientas: la calidad de la asistencia que el modelo puede ofrecer es proporcional a la calidad del contexto que el sistema le proporciona.

El problema que resuelve este capítulo

Un desarrollador que trabaja con IA en su IDE cotidiano experimenta, con regularidad, dos situaciones muy diferentes.

En la primera, el modelo sugiere exactamente la función que el desarrollador estaba escribiendo, en el estilo correcto, usando las convenciones del proyecto y llamando a las dependencias adecuadas. La sugerencia requiere ajustes mínimos y se integra sin fricción al codebase.

En la segunda, el modelo genera código que compila pero es incorrecto: usa una API que el proyecto no tiene, ignora las convenciones de nombres del equipo, reintroduce un bug que ya fue corregido tres semanas atrás o produce una solución que viola un requisito de negocio que estaba documentado en otro lugar.

La diferencia entre estas dos situaciones no está en el modelo. Está en el contexto. En el primer caso, el sistema le proporcionó al modelo información suficiente: el módulo donde se

insertará el código, las funciones relacionadas, las convenciones del proyecto, los tests existentes, la especificación de lo que debe hacer esa función. En el segundo caso, el modelo recibió solo la petición inmediata — "escribeme una función que haga X" — y generó la respuesta más plausible estadísticamente, que no es lo mismo que la respuesta correcta para ese proyecto específico.

Context Engineering para desarrollo de software es la disciplina de diseñar, gestionar y optimizar sistemáticamente ese contexto a lo largo de todo el ciclo de vida del software.

Lo que distingue este capítulo del Módulo 2

El Módulo 2 de este libro estudió el Prompt Engineering: cómo formular instrucciones claras, cómo usar few-shot examples, cómo encadenar prompts. Esas técnicas son necesarias y siguen siendo válidas. Pero el Context Engineering trasciende el prompt individual.

La diferencia es de nivel arquitectónico. El Prompt Engineering pregunta: "¿Cómo escribo este prompt?". El Context Engineering pregunta: "¿Qué información debe estar disponible en el contexto cuando el modelo ejecuta este prompt?". La segunda pregunta incluye la primera, pero además abarca decisiones sobre qué partes del repositorio recuperar, qué historia de commits es relevante, qué especificaciones incluir, cómo estructurar la memoria del agente que atraviesa múltiples sesiones de trabajo, y cómo integrar el output del modelo con las herramientas del pipeline de desarrollo.

En un sistema de asistencia al desarrollo de software de producción, el prompt que el desarrollador escribe es apenas la capa visible. Debajo hay una arquitectura de recuperación de contexto (qué código es relevante para esta petición), de gestión de estado (qué decidió el agente en pasos anteriores), de integración con herramientas (git log, linters, test runners) y de verificación (cómo se valida que el output es correcto antes de entregarlo al desarrollador). Todo eso es Context Engineering.

La estructura del capítulo

El capítulo sigue el ciclo de vida del software porque ese es el territorio donde el AI Engineer aplica sus decisiones de diseño. Cada fase del ciclo de vida plantea un problema de contexto diferente.

Bloque de fundamentos (secciones 01 y 02): qué es el Context Engineering aplicado a software, cómo el ciclo de vida del software impone requisitos de contexto específicos en cada etapa.

Bloque de fases del ciclo de vida (secciones 03 a 08): Context Engineering aplicado a análisis y relevamiento, diseño y arquitectura, generación de código, pruebas y aseguramiento de calidad, depuración y mantenimiento, e integración con IDEs, repositorios y pipelines de CI/CD.

Bloque de síntesis (secciones 09 a 13): patrones y anti-patrones del dominio, caso de estudio empresarial, laboratorio práctico, checklist del AI Engineer y resumen.

Bloque de cierre (secciones 14 y 15): autoevaluación y transición al capítulo 12.

El rol del modelo y el rol del profesional

Un principio transversal que atraviesa todo el capítulo: el modelo no reemplaza al arquitecto de software, al desarrollador sénior ni al tester experto. Amplifica su capacidad de trabajo.

Esta distinción no es retórica. Tiene consecuencias prácticas sobre cómo se diseña el sistema. Un sistema diseñado como si el modelo tomara decisiones autónomamente produce outputs que el equipo recibe como órdenes, con el consiguiente riesgo de integrar código incorrecto sin revisión crítica. Un sistema diseñado como herramienta de amplificación produce outputs que el profesional evalúa, ajusta y aprueba, con el modelo asumiendo el trabajo de generación y el humano asumiendo la responsabilidad de la decisión.

La arquitectura del contexto determina cuál de estos dos modos opera. Cuando el contexto incluye la especificación del problema, el código existente relacionado y los criterios de aceptación, el output del modelo es una propuesta técnica evaluable. Cuando el contexto está vacío o es mínimo, el output del modelo es una conjetura estadística que el profesional no tiene suficiente información para evaluar correctamente.

Nota del arquitecto

El mayor obstáculo práctico para implementar Context Engineering en un equipo de desarrollo no es técnico — es cultural. Los desarrolladores que adoptan herramientas de IA sin una comprensión de Context Engineering tienden a usar el modelo como un buscador de código mejorado: formulan preguntas puntuales y esperan respuestas correctas. Cuando las respuestas son incorrectas, concluyen que el modelo no es confiable.

Los equipos que implementan Context Engineering con rigor experimentan una transición diferente: el modelo pasa de ser un generador de código puntual a ser una capa del pipeline de desarrollo, con contexto estructurado, verificación sistemática y flujos de trabajo

reproducibles. La confiabilidad del sistema aumenta no porque el modelo sea mejor, sino porque el contexto está mejor diseñado.

La siguiente sección establece el mapa de ese territorio: cómo el ciclo de vida del software define los requisitos de contexto en cada una de sus fases.

Capítulo 11 — Context Engineering para Desarrollo de Software

Sección 02: El ciclo de vida del software asistido por IA

El ciclo de vida del software — el recorrido desde la captura de un requisito hasta el mantenimiento del sistema en producción — es el marco organizador de este capítulo. No porque el ciclo de vida sea el único modo de organizar el trabajo de desarrollo, sino porque cada fase plantea un problema de contexto diferente, y el AI Engineer necesita comprender esas diferencias antes de diseñar cualquier solución.

Esta sección establece el mapa conceptual del capítulo: qué ocurre en cada fase, qué tipo de contexto requiere la IA para asistir de manera útil en esa fase, y cuáles son las señales que indican que el contexto no está siendo diseñado correctamente.

Las fases del ciclo y sus demandas de contexto

El ciclo de vida del software moderno incluye seis fases principales desde la perspectiva de la asistencia con IA. Cada una tiene un perfil de contexto característico.

Análisis y relevamiento. El equipo trabaja con requisitos ambiguos, conversaciones con stakeholders, documentos de negocio y restricciones del sistema existente. La IA puede asistir en la estructuración y síntesis de esta información, en la identificación de contradicciones o ambigüedades, y en la generación de preguntas que el equipo no ha formulado aún. El contexto en esta fase es predominantemente texto de dominio: actas de reuniones, especificaciones funcionales, documentos de arquitectura existente, registros de decisiones. El reto es que este contexto suele ser extenso, inconsistente y no estructurado.

Diseño y arquitectura. El arquitecto trabaja con restricciones técnicas, patrones de diseño, capacidades del equipo y requisitos no funcionales. La IA puede asistir en la evaluación de alternativas de diseño, en la identificación de trade-offs, en la documentación de decisiones y en la verificación de consistencia entre el diseño propuesto y los principios establecidos del sistema. El contexto en esta fase incluye diagramas de arquitectura (traducidos a texto), ADRs

(Architecture Decision Records), documentación de APIs existentes y convenciones del proyecto.

Generación de código. El desarrollador traduce diseño en implementación. La IA puede generar código, completar funciones, sugerir implementaciones alternativas. El contexto en esta fase es el más técnico y el más preciso: el módulo donde se inserta el código, las interfaces que debe satisfacer, el estilo y las convenciones del proyecto, los tests que debe pasar, las funciones relacionadas existentes. Esta es la fase donde el impacto del contexto sobre la calidad del output es más inmediato y medible.

Pruebas y aseguramiento de calidad. El equipo de QA (o el mismo desarrollador) verifica que el código cumple los requisitos. La IA puede asistir en la generación de casos de prueba, en la identificación de escenarios no cubiertos, en el análisis de cobertura y en la revisión de pull requests. El contexto incluye la especificación funcional, el código bajo prueba, los tests existentes y las políticas de calidad del proyecto.

Depuración y mantenimiento. El desarrollador investiga comportamientos inesperados en el sistema existente. La IA puede asistir en el análisis de stack traces, en la hipótesis de causas raíz, en la revisión de cambios recientes relacionados y en la sugerencia de correcciones. El contexto incluye el error observado, el stack trace completo, el código de las funciones implicadas, el historial de cambios recientes y los tests que fallan.

Integración y despliegue. El pipeline automatiza la verificación y entrega del software. La IA puede asistir en la revisión de configuraciones de CI/CD, en el análisis de fallos de pipeline, en la generación de documentación de release y en la evaluación de impacto de cambios. El contexto incluye los archivos de configuración del pipeline, los logs de ejecución y las políticas de deployment.

El problema de la discontinuidad de contexto

Un patrón que aparece consistentemente en proyectos de software asistidos por IA es el de la discontinuidad de contexto: el modelo tiene acceso a información de una fase del ciclo pero no de las fases anteriores.

Un ejemplo concreto: un agente de generación de código que tiene acceso al código del módulo actual pero no a la especificación de diseño que determinó la estructura de ese módulo. El agente puede generar código que compila y pasa los tests inmediatos, pero que viola una restricción de diseño que solo estaba documentada en el ADR de la fase anterior. El

código generado es técnicamente correcto en el contexto estrecho que el agente ve, pero incorrecto en el contexto más amplio del proyecto.

Este problema no es un fallo del modelo. Es un problema de diseño del sistema: el arquitecto no estableció mecanismos para que el contexto de las fases anteriores esté disponible en las fases posteriores.

CICLO DE VIDA Y CONTEXTO ACUMULADO

```

Análisis → Diseño → Código → Pruebas → Debug → Deploy
    ↓           ↓           ↓           ↓           ↓           ↓
Actas        ADRs        Specs        Test cases  Traces    Config
Reqs        Diagramas    Style        Coverage   Changelogs  Logs
Restricc.    API docs    Tests        Políticas   Stack        Policies
  
```

CONTEXTO DISPONIBLE EN CADA FASE (sin diseño explícito):

Solo el artefacto inmediato de esa fase.

CONTEXTO DISPONIBLE CON DISEÑO EXPLÍCITO:

El artefacto de la fase actual + contexto seleccionado
de las fases anteriores relevantes para la tarea.

El principio de contexto acumulado selectivo

La solución no es incluir todo el contexto de todas las fases en cada llamada al modelo — eso agotaría la ventana de contexto y degradaría la calidad del razonamiento. La solución es diseñar el sistema para que recupere el contexto de fases anteriores de manera selectiva y relevante para la tarea específica.

Cuando el agente de generación de código trabaja en una función específica, el sistema debe ser capaz de recuperar: la especificación funcional de esa función (análisis), las restricciones de diseño que aplican al módulo (diseño), el estilo de código del proyecto (documentado en la fase de diseño o en un archivo de convenciones), y los tests existentes para funciones relacionadas (fase de pruebas anteriores). Esos cuatro elementos de contexto, combinados con la petición inmediata, producen un output radicalmente mejor que la petición sin contexto adicional.

El mecanismo para lograr esto varía según la escala del proyecto: puede ser tan simple como archivos de contexto pre-seleccionados por el desarrollador, o tan sofisticado como un sistema de RAG (Retrieval-Augmented Generation) que indexa todos los artefactos del proyecto y recupera los más relevantes en tiempo real. El principio es el mismo en ambos casos.

Herramientas del ciclo de vida y su rol en el contexto

Las herramientas que los equipos de desarrollo ya usan — control de versiones, gestores de issues, sistemas de documentación, pipelines de CI/CD — son fuentes naturales de contexto para la IA. El AI Engineer que diseña un sistema de asistencia al desarrollo aprovecha estas fuentes sin necesidad de crear repositorios de información adicionales.

El historial de commits de git es un registro de decisiones técnicas con timestamps. Los mensajes de commit bien escritos son el contexto de por qué el código cambió, no solo qué cambió. Las issues cerradas son el contexto de por qué ciertas funcionalidades existen. Los comentarios en pull requests son el contexto del razonamiento técnico del equipo. Los archivos de configuración del linter son el contexto de las convenciones de código que el proyecto adoptó.

Todas estas fuentes ya existen. El problema es que los sistemas de asistencia de IA muchas veces no las consultan. Diseñar el sistema para que las consulte selectivamente — según la tarea específica — es una de las decisiones más rentables que el AI Engineer puede tomar.

Nota del arquitecto

Al evaluar un proyecto de integración de IA en un equipo de desarrollo, la primera pregunta que hay que hacer no es "¿qué modelo vamos a usar?". La primera pregunta es "¿qué artefactos del ciclo de vida están disponibles como fuentes de contexto?". Si los mensajes de commit son descriptivos, si los ADRs están mantenidos, si las issues están bien documentadas, el sistema tiene una base rica para construir contexto de calidad. Si el repositorio tiene commits con mensajes como "fix" o "cambios", si no hay documentación de diseño, si las issues son epics sin subdividir, el sistema estará operando con contexto pobre independientemente de cuán sofisticado sea el modelo elegido.

La calidad de la ingeniería de software que el equipo practicaba antes de introducir IA determina, en gran medida, la calidad del contexto disponible para la IA. Invertir en Context Engineering y en prácticas de documentación del ciclo de vida son inversiones sinérgicas.

La siguiente sección comienza el análisis fase a fase. El primer punto de entrada es el análisis y relevamiento: cómo construir contexto útil para la IA a partir de la información más ambigua del ciclo de vida.

Capítulo 11 — Context Engineering para Desarrollo de Software

Sección 03: Contexto para análisis y relevamiento

La fase de análisis y relevamiento es donde el proyecto toma forma. El equipo trabaja con información heterogénea — conversaciones con usuarios, documentos de negocio, sistemas existentes, restricciones legales, objetivos estratégicos — y debe convertirla en especificaciones que el equipo técnico pueda implementar. Es la fase con mayor ambigüedad y, paradójicamente, donde las decisiones tienen mayor impacto en el costo total del proyecto.

La IA puede asistir en esta fase de maneras concretas y de alto valor. Pero solo si el contexto está bien diseñado. Esta sección describe cómo construir ese contexto.

El problema del análisis sin asistencia

Los analistas y product owners trabajan típicamente con grandes volúmenes de información: grabaciones de entrevistas, correos con stakeholders, documentos de requerimientos de versiones anteriores, reportes de problemas de usuarios y actas de reuniones. Sintetizar toda esa información en requisitos funcionales coherentes, sin contradicciones y con criterios de aceptación claros, es un trabajo que consume semanas.

La IA puede acelerar ese proceso — pero solo si tiene acceso al material de entrada. Un analista que le pregunta al modelo "¿cuáles son los requisitos para el módulo de pagos?" sin proporcionarle ningún documento previo recibirá una respuesta genérica sobre módulos de pagos típicos. Útil como punto de partida, pero no específica para el proyecto.

Las fuentes de contexto en la fase de análisis

El contexto para la IA en la fase de análisis proviene de cuatro fuentes principales:

Entrevistas y sesiones de relevamiento. Las transcripciones de entrevistas con usuarios y stakeholders son una fuente densa de información sobre requisitos reales. Cuando el modelo tiene acceso a estas transcripciones, puede identificar patrones repetidos ("el stakeholder A y el stakeholder C mencionan el mismo problema de tres formas diferentes"), contradicciones

("el departamento de ventas quiere X, el departamento de operaciones quiere lo opuesto"), y preguntas sin respuesta ("nadie mencionó cómo manejar el caso en que el usuario no tiene conexión").

Documentos de negocio. Contratos de servicio, regulaciones aplicables, políticas internas, documentación del sistema legado — todos estos son restricciones que el sistema debe respetar. La IA, con acceso a estos documentos, puede verificar que las especificaciones que el equipo está generando no violen restricciones que están documentadas pero que el analista puede haber pasado por alto.

Sistema existente. Si hay un sistema previo (legado o versión anterior), el código y la documentación de ese sistema son contexto invaluable. El modelo puede analizar el sistema existente para identificar funcionalidades implícitas que no están documentadas pero que los usuarios conocen y esperan que se preserven.

Decisiones anteriores. Los ADRs y registros de decisiones de proyectos anteriores del mismo equipo o dominio son contexto estratégico: qué alternativas fueron evaluadas y descartadas, por qué razones, con qué restricciones. Evitan que el equipo repierta debates que ya tuvo.

Cómo estructurar el contexto de análisis

La ventana de contexto de cualquier modelo tiene límites. Para proyectos grandes, la suma de todas las fuentes anteriores excede con creces esa ventana. El AI Engineer necesita una estrategia de selección.

El principio general es el de relevancia para la tarea específica. No se incluye "toda la documentación del proyecto" en el contexto — se incluye la documentación relevante para la tarea que se está ejecutando en ese momento.

TAREA: Identificar contradicciones en los requisitos del módulo de notificaciones

CONTEXTO RELEVANTE:

- ✓ Acta de reunión con el equipo de marketing (mencionan notificaciones push)
- ✓ Acta de reunión con el equipo de legal (restricciones de consentimiento GDPR)
- ✓ Issues del sistema actual relacionadas con notificaciones
- ✓ Especificación funcional de notificaciones v1.0 (versión anterior)

CONTEXTO NO INCLUIDO:

- ✗ Actas de reuniones sobre el módulo de pagos (diferente dominio)
- ✗ Contratos de proveedores de infraestructura (no relevante para esta tarea)
- ✗ Código del sistema legado (para análisis, no es la fuente primaria)

Este esquema de selección puede ser manual — el analista selecciona los documentos relevantes antes de la sesión — o automatizado mediante un sistema de RAG que indexa todos los artefactos del proyecto y recupera los más similares semánticamente a la tarea.

El modelo como amplificador del analista, no como reemplazo

Es importante establecer con precisión el rol del modelo en la fase de análisis. El modelo no reemplaza al analista. No tiene el juicio de negocio, el conocimiento tácito de la organización ni la capacidad de evaluar la viabilidad política de una especificación. Lo que puede hacer es amplificar la capacidad del analista en tres dimensiones:

Síntesis de volumen. El analista puede darle al modelo 50 páginas de transcripciones y pedirle que identifique los temas recurrentes, las contradicciones y las preguntas sin respuesta. Esto reduce semanas de trabajo manual a horas.

Exhaustividad. El modelo, dado un conjunto de requisitos, puede generar sistemáticamente las preguntas que el analista debería hacer para completarlos: "¿Qué ocurre si el usuario no tiene email verificado?", "¿Cómo se maneja el timeout de la sesión?", "¿Quién tiene permiso para aprobar esta acción?". El analista evalúa cuáles de esas preguntas son relevantes para el proyecto.

Consistencia. El modelo puede verificar si un conjunto de requisitos es internamente consistente: si el requisito A dice que solo usuarios administradores pueden hacer X, y el requisito B dice que cualquier usuario puede hacer X, hay una contradicción que el modelo puede identificar aunque esté en páginas separadas del documento.

Ejemplo concreto: estructurar requisitos desde entrevistas

Supóngase que el equipo tiene cuatro transcripciones de entrevistas sobre un sistema de gestión de tickets de soporte. El analista construye el siguiente contexto para el modelo:

CONTEXTO:

[Transcripción entrevista 1 - Agente de soporte senior]

[Transcripción entrevista 2 - Supervisor de soporte]

[Transcripción entrevista 3 - Cliente corporativo]

[Transcripción entrevista 4 - Director de operaciones]

REQUISITOS ACTUALES BORRADORES:

[5 requisitos que el analista ya tiene documentados]

TAREA:

Basándote en las transcripciones y los requisitos borradores:

1. Identifica requisitos implícitos que los entrevistados mencionan pero que no están en los borradores.
2. Identifica contradicciones entre lo que dijeron distintos entrevistados.
3. Lista las 10 preguntas más importantes que el equipo debería responder antes de cerrar el análisis.

El output de este prompt, con contexto bien construido, puede ahorrarle al analista dos o tres días de trabajo de síntesis. Sin el contexto de las transcripciones, el output sería genérico e inútil.

Artefactos de salida del análisis como contexto para fases posteriores

Los artefactos que produce la fase de análisis — especificaciones funcionales, user stories, criterios de aceptación, restricciones documentadas — se convierten en contexto para todas las fases posteriores. Este es el primer eslabón de la cadena de contexto acumulado.

Si los criterios de aceptación están bien formulados en esta fase, la fase de pruebas tiene un contexto claro para la generación de casos de test. Si las restricciones de negocio están explícitamente documentadas, la fase de diseño tiene el contexto necesario para evaluar alternativas de arquitectura. Si las user stories incluyen casos borde, la fase de generación de código tiene el contexto para manejar esos casos.

La calidad del análisis y la calidad del contexto que produce para las fases posteriores son la misma cosa, vista desde ángulos distintos.

Nota del arquitecto

El riesgo más alto en el uso de IA en la fase de análisis no es que el modelo genere especificaciones incorrectas — esas se verifican con los stakeholders. El riesgo más alto es que el equipo tome las especificaciones generadas por IA como un primer borrador incompleto y no las someta a la validación con usuarios reales. La IA puede identificar preguntas que el analista no hizo, pero no puede responderlas. Solo los stakeholders pueden hacerlo.

El flujo correcto es: IA genera borrador a partir del contexto disponible → analista revisa y valida con stakeholders → diferencias entre borrador y validación revelan puntos de ambigüedad → IA ayuda a resolverlos → ciclo continúa hasta que el análisis está completo.

La siguiente sección avanza a la fase de diseño y arquitectura: cómo construir el contexto para que la IA asista en decisiones técnicas sin suplantar el juicio del arquitecto de software.

Capítulo 11 — Context Engineering para Desarrollo de Software

Sección 04: Contexto para diseño y arquitectura

La fase de diseño es donde el arquitecto de software toma las decisiones más difíciles del proyecto: qué estructura tendrá el sistema, cómo se comunicarán sus componentes, qué patrones se aplicarán, qué restricciones no funcionales determinan las elecciones técnicas. Estas decisiones tienen efectos de largo plazo — el costo de revertirlas una vez que el código está escrito puede ser enorme.

La IA puede asistir en esta fase de formas valiosas, pero opera bajo una limitación fundamental que el arquitecto debe entender claramente: el modelo no tiene juicio de negocio propio, no conoce el contexto organizacional del equipo y no evaluará correctamente las restricciones tácitas que solo el arquitecto conoce. Lo que puede hacer es amplificar el razonamiento del arquitecto cuando se le proporciona el contexto correcto.

Qué aporta la IA al diseño arquitectónico

La asistencia de IA en diseño y arquitectura es valiosa en cuatro áreas específicas:

Evaluación de alternativas. Dado un problema de diseño con restricciones explícitas, el modelo puede describir las alternativas de solución conocidas, con sus trade-offs, precedentes de uso y condiciones bajo las cuales cada una es preferible. El arquitecto no usa esto como un oráculo que le dice qué hacer — lo usa como un sistema de consulta que le muestra el espacio de soluciones antes de tomar su decisión.

Verificación de consistencia. Una vez que el arquitecto tiene un diseño propuesto, el modelo puede verificar si ese diseño es consistente con los principios y restricciones documentados del proyecto. Si el ADR número 7 establece que el sistema no debe tener dependencias directas entre el módulo A y el módulo B, y el diseño propuesto introduce una dependencia de ese tipo, el modelo puede identificarlo — si esos ADRs están en el contexto.

Documentación de decisiones. Los ADRs son artefactos de alta calidad que los equipos producen con baja frecuencia, en parte porque documentarlos correctamente es costoso en tiempo. Con el contexto de la discusión de diseño y la decisión tomada, el modelo puede generar un borrador de ADR que el arquitecto revisa y aprueba. Esto reduce el costo de documentar sin reducir la calidad.

Identificación de riesgos. Dado un diseño propuesto, el modelo puede identificar patrones de riesgo conocidos: acoplamientos que pueden crear bottlenecks, decisiones que limitan la escalabilidad futura, dependencias de terceros con historial de breaking changes. El arquitecto evalúa cuáles de estos riesgos son relevantes en su contexto específico.

El contexto necesario para asistencia en diseño

La calidad de la asistencia en diseño depende directamente de qué contexto recibe el modelo. Los elementos mínimos necesarios son:

Restricciones del sistema. Los requisitos no funcionales del proyecto: latencia máxima, throughput esperado, presupuesto de infraestructura, restricciones de seguridad, regulaciones aplicables. Sin estas restricciones, el modelo generará diseños académicamente correctos pero posiblemente inviables en el contexto del proyecto.

Principios arquitectónicos establecidos. Si el equipo adoptó decisiones arquitectónicas previas — microservicios vs. monolito, base de datos centralizada vs. distribuida, patrones de messaging — estas decisiones deben estar en el contexto. El modelo debe saber que está trabajando dentro de un sistema existente con principios ya establecidos, no diseñando desde cero.

ADRs relevantes. Los Architecture Decision Records de decisiones relacionadas con el problema actual. Si el equipo ya decidió que usa event sourcing para el dominio de pedidos, y ahora está diseñando el dominio de inventario, los ADRs de pedidos son contexto relevante.

El problema específico. La descripción precisa del problema de diseño que se está resolviendo: qué módulo, qué interacciones, qué restricciones locales, qué casos de uso debe satisfacer.

ESTRUCTURA DE CONTEXTO PARA SESIÓN DE DISEÑO

[PRINCIPIOS ARQUITECTÓNICOS DEL PROYECTO]

- Arquitectura: microservicios con API Gateway
- Comunicación: REST para sincrónico, eventos (Kafka) para asincrónico
- Base de datos: una base por servicio, sin compartir esquemas
- Lenguaje: Python 3.11+, FastAPI

[ADRS RELEVANTES]

- ADR-003: Decisión de usar Kafka para notificaciones
- ADR-007: Prohibición de dependencias directas entre servicios

[RESTRICCIONES NO FUNCIONALES]

- Latencia máxima para checkout: 300ms P95
- Disponibilidad requerida: 99.9%
- Compliance: PCI-DSS para datos de pago

[PROBLEMA A DISEÑAR]

Diseñar el servicio de reconciliación de pagos que debe:

- Recibir eventos de pagos procesados
- Verificar contra registros del banco
- Emitir alertas para discrepancias
- Generar reportes diarios

Con este contexto, el modelo puede razonar sobre el diseño del servicio respetando los principios y restricciones del sistema real, no un sistema hipotético.

El arquitecto decide, el modelo asiste

Hay una diferencia cualitativa entre las fases de análisis y diseño en cuanto al rol del modelo. En análisis, el trabajo principal es de síntesis de información: el modelo puede procesar grandes volúmenes de texto y extraer patrones. En diseño, el trabajo principal es de evaluación y juicio: cuál alternativa es mejor para este equipo, este proyecto y esta organización.

La evaluación y el juicio requieren información que el modelo generalmente no tiene y no puede tener en el contexto: las capacidades reales del equipo, la deuda técnica acumulada, las

restricciones políticas de la organización, la visión de largo plazo del producto, los compromisos con clientes específicos. El arquitecto tiene acceso a toda esa información. El modelo no.

El error de diseño más frecuente es tratar al modelo como árbitro de decisiones arquitectónicas. El flujo correcto es el inverso: el arquitecto formula la decisión que considera correcta, proporciona al modelo el contexto del problema y la decisión propuesta, y le pide que identifique riesgos, casos no considerados o alternativas que el arquitecto no evaluó. El arquitecto usa ese output como insumo para revisar su decisión, no como sustituto de ella.

Generación de diagramas y documentación

Una de las aplicaciones más inmediatamente útiles de la IA en diseño es la asistencia en documentación. Los arquitectos suelen tener el diseño claro en su cabeza pero posterguen documentarlo porque es costoso en tiempo.

El modelo puede asistir en la traducción del diseño a distintos formatos de documentación:

- A partir de una descripción en lenguaje natural del diseño, generar la especificación en formato de diagrama (PlantUML, Mermaid, C4 model en texto)
- A partir de la discusión del equipo sobre una decisión, generar el borrador del ADR con la estructura estándar (contexto, decisión, consecuencias)
- A partir del diseño aprobado, generar la documentación de API en formato OpenAPI
- A partir de los diagramas de secuencia, generar los contratos entre servicios

En todos estos casos, el modelo trabaja con el contexto del diseño aprobado para producir artefactos de documentación que el arquitecto revisa y ajusta. El costo de documentación se reduce significativamente sin comprometer la calidad, porque el arquitecto evalúa el output antes de aprobarlo.

Validación del diseño contra el análisis

Un uso de alto valor que suele pasarse por alto es la verificación de que el diseño propuesto cubre todos los requisitos del análisis. Con las especificaciones funcionales y el diseño propuesto en el contexto, el modelo puede identificar requisitos funcionales que no tienen un componente de diseño correspondiente, o componentes de diseño que no responden a ningún requisito documentado.

Este tipo de verificación es tedioso de hacer manualmente y fácil de omitir bajo presión de tiempo. La IA puede ejecutarlo sistemáticamente como un paso de validación antes de que el

equipo avance a la generación de código.

Nota del arquitecto

La trampa más sutil en el uso de IA para diseño arquitectónico es la de la coherencia superficial: el modelo puede generar diseños que parecen técnicamente sólidos pero que en realidad son compilaciones de patrones tomados de contextos diferentes. Un diseño que usa event sourcing para el módulo A, CQRS para el módulo B y una base de datos compartida para el módulo C puede parecerle coherente al modelo (cada patrón es válido en sí mismo) pero ser arquitectónicamente inconsistente para el sistema completo.

La verificación de coherencia arquitectónica global requiere el juicio del arquitecto, que conoce el sistema como un todo. El modelo puede identificar inconsistencias locales si el contexto de los principios globales está disponible, pero no puede sustituir la visión de conjunto que solo el arquitecto tiene.

La siguiente sección entra en la fase de generación de código — el corazón del capítulo desde la perspectiva del impacto inmediato — y analiza en profundidad cómo el contexto transforma la calidad del código generado.

Capítulo 11 — Context Engineering para Desarrollo de Software

Sección 05: Contexto para generación de código

Esta es la sección más crítica del capítulo. La generación de código es la aplicación de IA para desarrollo de software de mayor adopción actual, y también donde el impacto del Context Engineering es más inmediato y medible. Un experimento que cualquier desarrollador puede replicar en minutos demuestra el punto con claridad.

El experimento del contexto

Tómese la siguiente tarea de generación de código:

"Escribe una función que calcule el precio total de un pedido aplicando descuentos."

Esta misma tarea, enviada al mismo modelo, produce resultados radicalmente diferentes según el contexto que la acompaña.

Caso 1: Sin contexto

```
# Output del modelo sin contexto adicional

def calcular_precio_total(items, descuento=0):
    """
    Calcula el precio total de un pedido aplicando un descuento.

    Args:
        items: lista de diccionarios con 'precio' y 'cantidad'
        descuento: porcentaje de descuento a aplicar (0-100)

    Returns:
        float: precio total con descuento aplicado
    """
    subtotal = sum(item['precio'] * item['cantidad'] for item in items)
    precio_final = subtotal * (1 - descuento / 100)
    return round(precio_final, 2)
```

El código es correcto en abstracto. Pero no sabe nada del proyecto: no usa las convenciones de naming, no usa las clases de dominio existentes, no maneja los tipos de descuento que el negocio tiene definidos, no lanza las excepciones que el proyecto usa, no incluye logging.

Caso 2: Con contexto de módulo y convenciones

CONTEXTO PROPORCIONADO:

Módulo: `orders/services/pricing.py`

Convenciones del proyecto:

- Type hints obligatorios en todas las funciones
- Docstrings en formato Google Style
- Manejo de errores con excepciones custom de `orders/exceptions.py`
- Logging con `structlog`

Clases disponibles en `orders/models.py`:

```
class Order:
    items: list[OrderItem]
    customer_id: UUID

class OrderItem:
    product_id: UUID
    quantity: int
    unit_price: Decimal

class DiscountRule:
    type: Literal['percentage', 'fixed', 'buy_x_get_y']
    value: Decimal
    min_order_amount: Optional[Decimal]
```

Excepciones disponibles en `orders/exceptions.py`:

```
class InvalidDiscountError(OrderError): ...

class OrderCalculationError(OrderError): ...
```

Función de pricing existente como referencia:

```
def calculate_tax(order: Order, tax_rate: Decimal) -> Decimal:
    # [código de la función existente]
```

Con este contexto, el output del modelo es sustancialmente diferente:

```
# Output del modelo con contexto completo

import structlog

from decimal import Decimal

from orders.models import Order, DiscountRule

from orders.exceptions import InvalidDiscountError, OrderCalculationError

logger = structlog.get_logger(__name__)

def calculate_order_total(
    order: Order,
    discount_rules: list[DiscountRule],
) -> Decimal:
    """Calcula el precio total del pedido aplicando reglas de descuento.

    Args:
        order: Instancia del pedido con sus ítems.
        discount_rules: Lista de reglas de descuento a aplicar en orden.

    Returns:
        Precio total con todos los descuentos aplicados.

    Raises:
        InvalidDiscountError: Si alguna regla de descuento tiene valores inválidos.
        OrderCalculationError: Si el cálculo falla por datos inconsistentes.
    """
    logger.info("calculating_order_total", order_id=str(order.id))

    try:
        subtotal = sum(
            item.unit_price * item.quantity
            for item in order.items
        )
    except Exception as e:
```

```
        raise OrderCalculationError(
            f"Error calculando subtotal del pedido {order.id}"
        ) from e

    total = subtotal

    for rule in discount_rules:
        if rule.min_order_amount and subtotal < rule.min_order_amount:
            continue # El pedido no alcanza el mínimo para este descuento

        if rule.type == 'percentage':
            if not (Decimal('0') <= rule.value <= Decimal('100')):
                raise InvalidDiscountError(
                    f"Porcentaje de descuento inválido: {rule.value}"
                )
            total -= total * (rule.value / Decimal('100'))

        elif rule.type == 'fixed':
            total = max(Decimal('0'), total - rule.value)

        else:
            raise InvalidDiscountError(
                f"Tipo de descuento no soportado: {rule.type}"
            )

    logger.info(
        "order_total_calculated",
        order_id=str(order.id),
        subtotal=str(subtotal),
        total=str(total),
    )

    return total
```

La diferencia no es cosmética. El segundo código es directamente integrable al proyecto. El primero requiere reescritura significativa.

Los elementos del contexto para generación de código

El experimento anterior ilustra los cinco elementos de contexto que determinan la calidad del código generado:

1. Ubicación en el repositorio. El módulo, el paquete, el archivo donde se va a insertar el código. Esta información le dice al modelo el dominio, el nivel de abstracción y el propósito del código.

2. Clases y tipos del dominio. Las estructuras de datos del proyecto con las que el código debe trabajar. Sin esto, el modelo inventa sus propias estructuras que luego hay que integrar manualmente.

3. Convenciones del proyecto. Estilo de código, manejo de errores, logging, formato de documentación. Sin esto, el código generado es técnicamente correcto pero estilísticamente inconsistente, lo que genera fricción en la revisión de código.

4. Funciones relacionadas existentes. Las funciones que el código nuevo va a llamar o que lo llamarán. Sin esto, el modelo puede generar código que asume interfaces que no existen o que ignora funciones de utilidad que ya están disponibles.

5. Tests existentes. Los tests que el código debe pasar. Con tests en el contexto, el modelo puede generar código que directamente satisface esos tests, en lugar de código que el desarrollador luego debe ajustar para que los tests pasen.

El rol de los tests como contexto

Este último elemento merece atención especial porque invierte la lógica habitual. En lugar de generar código y luego escribir tests, el workflow asistido por IA más productivo es:

1. El desarrollador escribe los tests primero (o tiene tests existentes para la función que está modificando).
2. Los tests se incluyen en el contexto del prompt de generación.
3. El modelo genera la implementación que satisface esos tests.
4. El desarrollador ejecuta los tests para verificar.

Este workflow — variante del Test-Driven Development — es especialmente efectivo con IA porque los tests son una especificación ejecutable precisa. El modelo no necesita inferir qué debe hacer la función: los tests se lo dicen.

WORKFLOW TDD-ASISTIDO POR IA

[PASO 1] Desarrollador escribe tests:

```
def test_calculate_order_total_with_percentage_discount():  
    order = Order(items=[OrderItem(unit_price=100, quantity=2)])  
    rule = DiscountRule(type='percentage', value=10)  
    result = calculate_order_total(order, [rule])  
    assert result == Decimal('180.00')  
  
def test_calculate_order_total_skips_rule_below_minimum():  
    order = Order(items=[OrderItem(unit_price=50, quantity=1)])  
    rule = DiscountRule(type='percentage', value=20, min_order_amount=100)  
    result = calculate_order_total(order, [rule])  
    assert result == Decimal('50.00') # Descuento no aplica
```

[PASO 2] Tests + contexto del módulo → modelo genera implementación

[PASO 3] Desarrollador ejecuta tests → verifica que pasan

[PASO 4] Desarrollador revisa el código → aprueba o ajusta

Estrategias de selección de contexto para proyectos grandes

En proyectos grandes, incluir todo el código relacionado en el contexto no es práctico. El repositorio puede tener decenas de miles de archivos. La selección de contexto requiere estrategia.

Selección manual guiada. El desarrollador selecciona explícitamente los archivos relevantes: el módulo donde trabaja, las clases que va a usar, la función de referencia más similar a la que va a escribir. Herramientas como Cursor permiten hacer esta selección de forma visual.

Recuperación automática por embeddings. Un sistema de RAG indexa el repositorio por embeddings semánticos. Cuando el desarrollador formula la tarea, el sistema recupera automáticamente los fragmentos de código más similares semánticamente. Efectivo para repositorios grandes, pero requiere infraestructura de indexación.

Archivos de contexto del proyecto. Un archivo `CONTEXT.md` o `.claude/project.md` en el repositorio que documenta las convenciones, los patrones estándar del proyecto y las instrucciones para el asistente de IA. Este archivo, incluido automáticamente en cada sesión, establece el contexto base del proyecto sin necesidad de selección manual por tarea.

Code review asistido por IA

La generación de código tiene un paso posterior que también se beneficia del Context Engineering: la revisión de código. Cuando un pull request incluye en el contexto el diff, las guías de estilo del proyecto, los requisitos que motivaron el cambio y el código de las funciones afectadas, el modelo puede hacer una revisión técnica más precisa que una revisión sin contexto.

El contexto para code review incluye:

- El diff completo del pull request
- El contexto de las funciones modificadas (no solo las líneas cambiadas)
- Los requisitos o issues que motivaron el cambio
- Las guías de estilo y convenciones del proyecto
- Los tests que se agregaron o modificaron

Con este contexto, el modelo puede identificar: casos borde no manejados, violaciones de convenciones del proyecto, inconsistencias con el diseño existente, riesgos de regresión en funciones relacionadas, y documentación faltante o incorrecta.

Nota del arquitecto

El mayor riesgo en la generación de código asistida por IA no es que el modelo genere código incorrecto — eso lo detectan los tests y la revisión. El mayor riesgo es que el equipo adopte el código generado sin revisión crítica, confiando en que el modelo verificó su propia corrección. El modelo no puede verificar su propia corrección. Puede producir código que compila, que pasa los tests inmediatos y que aun así es incorrecto respecto a los requisitos de negocio que no estaban en el contexto.

El flujo de trabajo correcto siempre incluye: generación → verificación automática (tests, linter) → revisión humana → aprobación. La IA acelera la generación; el humano sigue siendo responsable de la corrección.

La siguiente sección analiza la fase de pruebas y aseguramiento de calidad: cómo el Context Engineering amplía la cobertura de tests y mejora la detección de defectos.

Capítulo 11 — Context Engineering para Desarrollo de Software

Sección 06: Contexto para pruebas y aseguramiento de calidad

Las pruebas son el mecanismo principal de verificación de que el software hace lo que se supone que debe hacer. También son, en muchos equipos, el artefacto más costoso de producir en relación a su impacto: escribir buenos casos de prueba requiere una comprensión profunda de los requisitos, del código bajo prueba y de los escenarios de falla posibles.

La IA puede asistir de manera significativa en esta fase — pero, como en las anteriores, la calidad de la asistencia depende del contexto disponible. Esta sección analiza cómo construir ese contexto.

Los tres problemas que la IA resuelve en testing

Cobertura incompleta. Los desarrolladores tienden a escribir tests para el caso feliz y los casos de error obvios. Los casos borde — condiciones límite, combinaciones inusuales de parámetros, comportamientos de sistema bajo carga — frecuentemente no tienen tests porque requieren esfuerzo adicional para identificarlos. El modelo, dado el código de la función y su especificación, puede generar sistemáticamente casos borde que el desarrollador no consideró.

Eficiencia de escritura. Escribir tests bien estructurados — con setup, assertions claras, nombres descriptivos — es trabajo repetitivo. El modelo puede generar la estructura de los tests a partir de los casos de prueba que el desarrollador identifica, reduciendo el tiempo de escritura.

Revisión de tests existentes. Los tests antiguos pueden ser obsoletos (verifican comportamientos que ya cambiaron), redundantes (múltiples tests verifican lo mismo) o incompletos (no verifican el comportamiento actual de la función). El modelo puede revisar

un conjunto de tests y señalar estos problemas cuando tiene el contexto del código actual y la especificación vigente.

El contexto para generación de tests

El contexto para la generación de tests de calidad tiene cinco componentes:

La función o módulo bajo prueba. El código que se quiere testear, completo. Esto es obvio, pero frecuentemente los desarrolladores proporcionan solo la firma de la función y esperan que el modelo infiera el comportamiento. El modelo genera mejores tests cuando puede analizar la implementación real.

La especificación funcional. Los requisitos o user stories que definen qué debe hacer el código. Sin esto, el modelo solo puede generar tests que verifican la implementación actual, no tests que verifican que la implementación cumple los requisitos. La diferencia es crítica: un test que verifica la implementación siempre pasa (valida lo que el código hace), pero puede no detectar un bug donde el código hace algo diferente a lo especificado.

Los tests existentes del módulo. Para no duplicar cobertura y para mantener el estilo y las convenciones de testing del proyecto. Si el proyecto usa fixtures de pytest de cierta manera, los nuevos tests deben seguir el mismo patrón.

Las dependencias y sus mocks. Si la función usa base de datos, APIs externas u otros servicios, el modelo necesita saber cómo esas dependencias están mockeadas en el proyecto, o sugerirá mocks que son incompatibles con la infraestructura de testing existente.

Criterios de aceptación explícitos. Los casos de prueba que el equipo ya identificó como obligatorios. El modelo los incluye en los tests generados y agrega casos adicionales para completar la cobertura.

Ejemplo: generación de tests con contexto completo

```
CONTEXTO PARA GENERACIÓN DE TESTS:

[FUNCIÓN BAJO PRUEBA]

def calculate_order_total(
    order: Order,
    discount_rules: list[DiscountRule],
) -> Decimal:
    # [implementación completa]

[ESPECIFICACIÓN FUNCIONAL]

- El total debe calcularse como la suma de (precio_unitario * cantidad)
  para todos los ítems del pedido.

- Las reglas de descuento se aplican en el orden en que se proveen.

- Si el subtotal no alcanza el monto mínimo de una regla, esa regla
  se omite sin error.

- Los descuentos de tipo 'percentage' reducen el total en ese porcentaje.

- Los descuentos de tipo 'fixed' reducen el total en esa cantidad fija,
  con un piso de 0 (el total no puede ser negativo).

- Si el valor de un descuento percentage está fuera del rango [0, 100],
  debe lanzarse InvalidDiscountError.

[TESTS EXISTENTES DEL MÓDULO]

# [contenido de tests/orders/test_pricing.py existente]

[MOCKS DISPONIBLES]

# El proyecto usa pytest-mock y factory_boy para fixtures
# Ver conftest.py para las factories de Order y OrderItem

TAREA: Genera tests que cubran los casos borde de la función,
       además de los casos especificados. Usa el estilo del
       proyecto (factory_boy para fixtures, pytest para assertions).
```

Con este contexto, el modelo genera tests que cubren: pedido vacío, un solo ítem, múltiples ítems, descuento no aplicable por monto mínimo, múltiples descuentos apilados, descuento

que llevaría el total a negativo, y valores inválidos de descuento — casos que el desarrollador podría haber pasado por alto.

Testing de propiedades y mutación

Un uso avanzado de IA en testing que el Context Engineering habilita es la generación de property-based tests y la sugerencia de operadores de mutación para evaluación de la calidad del test suite.

Los property-based tests, a diferencia de los tests de ejemplo, verifican propiedades generales del código para un espacio amplio de entradas generadas automáticamente. Son especialmente útiles para funciones matemáticas o de transformación de datos.

El modelo puede sugerir propiedades relevantes para una función dada su especificación:

```
PROPIEDAD 1: El total con descuento siempre es <= al subtotal sin descuento.  
PROPIEDAD 2: Si el descuento es 0%, el total es igual al subtotal.  
PROPIEDAD 3: Aplicar dos reglas de descuento en orden diferente puede  
                producir totales diferentes (el orden importa).  
PROPIEDAD 4: El total nunca es negativo, incluso con descuentos fixed grandes.
```

Estas propiedades, identificadas por el modelo a partir de la especificación, se convierten en tests de Hypothesis (Python) o QuickCheck (Haskell/Scala) que el desarrollador configura para exploración automática.

IA para revisión de pull requests

La revisión de pull requests es una de las aplicaciones de mayor impacto empresarial de la IA en QA, y está directamente relacionada con el Context Engineering. El valor de una revisión automática depende casi completamente de la calidad del contexto disponible.

Una revisión con contexto mínimo (solo el diff) puede identificar: problemas de sintaxis, violaciones de linting, variables no usadas. Útil, pero replicable por herramientas estáticas existentes.

Una revisión con contexto completo puede identificar: casos borde no cubiertos por los nuevos tests, inconsistencias con las guías de diseño del proyecto, regresiones potenciales en funciones que llaman al código modificado, violaciones de invariantes de negocio documentados en la especificación, y documentación faltante o incorrecta.


```
CONTEXTO PARA REVISIÓN DE PULL REQUEST:

[DIFF DEL PR]

# [contenido del git diff]

[CONTEXTO DE FUNCIONES AFECTADAS]

# [código completo de las funciones modificadas y las que las llaman]

[REQUISITOS QUE MOTIVARON EL CAMBIO]

# Issue #342: El cálculo de descuentos no respeta el monto mínimo

[GUÍAS DE ESTILO DEL PROYECTO]

# [extracto del documento de convenciones]

[TESTS DEL MÓDULO AFECTADO]

# [tests existentes y nuevos tests del PR]

TAREA: Revisa el PR con foco en:

1. ¿Los nuevos tests cubren los casos de la issue #342?
2. ¿Hay casos borde de la especificación sin test?
3. ¿El código sigue las convenciones del proyecto?
4. ¿Hay riesgos de regresión en el código que usa esta función?
```

Testing no funcional

La IA también puede asistir en la generación de tests de performance, seguridad y resiliencia cuando el contexto incluye las restricciones no funcionales del sistema.

Por ejemplo, con los requisitos de latencia del sistema ($P_{95} < 300\text{ms}$ para checkout), el modelo puede generar scripts de carga usando k6 o Locust que simulan la carga esperada y verifican que el sistema cumple esas restricciones. Sin las restricciones no funcionales en el contexto, el modelo generaría scripts de carga genéricos que no corresponden a los objetivos reales del sistema.

Nota del arquitecto

Un error frecuente al implementar IA para generación de tests es confundir cobertura de líneas con calidad de tests. El modelo puede generar tests que aumentan la métrica de cobertura sin agregar valor real de verificación — por ejemplo, tests que llaman a una función y solo verifican que no lanza excepciones, sin verificar el output. El ingeniero de QA debe evaluar la calidad de los tests generados, no solo su cantidad.

La métrica correcta no es "cuántos tests generó el modelo" sino "cuántos bugs adicionales detectarían estos tests que los tests anteriores no detectaban". Esa evaluación requiere juicio humano.

La siguiente sección aborda la fase de depuración y mantenimiento: cómo construir el contexto de diagnóstico para que la IA asista en la identificación de causas raíz de manera efectiva.

Capítulo 11 — Context Engineering para Desarrollo de Software

Sección 07: Contexto para depuración y mantenimiento

La depuración es la fase del ciclo de vida del software donde el Context Engineering tiene el mayor impacto en términos de tiempo ahorrado. Encontrar la causa raíz de un bug puede llevar horas o días cuando el desarrollador trabaja solo con su conocimiento del sistema y un stack trace. Con el contexto correcto, el modelo puede proponer hipótesis de causa raíz bien fundamentadas en minutos.

Pero el contexto correcto para debugging no es simplemente "el error que apareció". Es el conjunto de información que un desarrollador sénior experimentado necesitaría para analizar el problema: el stack trace completo, el código de las funciones implicadas, el historial de cambios recientes, los tests que fallan, el comportamiento observado versus el esperado.

El problema del debugging sin contexto

El caso más común de debugging mal asistido por IA es este: el desarrollador pega el stack trace en el chat del modelo y pregunta "¿qué está fallando?". El modelo lee el stack trace y genera una explicación del tipo de error — "parece un NullPointerException en la línea X" — que no agrega información que el desarrollador no tenía ya.

La razón es que el stack trace solo describe el síntoma, no la causa. El modelo no puede inferir la causa sin el contexto del código que generó ese error, el estado del sistema cuando ocurrió y los cambios recientes que podrían haberlo introducido.

El debugging asistido por IA es valioso cuando el contexto incluye todo lo que un desarrollador sénior necesitaría para diagnosticar el problema.

El contexto de diagnóstico completo

El contexto para debugging efectivo tiene seis componentes:

El error observado. No solo el mensaje de error, sino la descripción del comportamiento observado versus el esperado. "La función devuelve None cuando debería devolver el objeto Order" es más útil que "Error en pricing.py línea 47".

El stack trace completo. No el stack trace truncado que aparece en los logs de producción, sino el completo, con todos los frames, valores de variables locales si están disponibles, y el contexto de ejecución (request ID, usuario, timestamp).

El código de las funciones implicadas. Las funciones que aparecen en el stack trace, completas. No solo las líneas mencionadas en el trace, sino el código completo de cada función para que el modelo pueda razonar sobre el flujo de control.

El historial de cambios recientes. El output de `git log --oneline -20` y el diff de los cambios más recientes en los módulos afectados. Si el bug apareció después de un deploy específico, el diff de ese deploy es contexto crítico.

Los tests que fallan. Si el bug triggerea fallos en el test suite, los tests fallidos con su output completo son contexto adicional que describe el comportamiento incorrecto de manera ejecutable.

La configuración del entorno. Variables de entorno, versiones de dependencias, configuración de base de datos — especialmente si el bug se manifiesta en producción pero no en desarrollo.

Construcción del contexto de diagnóstico: ejemplo práctico

Supóngase que el sistema de cálculo de precios comienza a devolver totales incorrectos en producción. El desarrollador construye el siguiente contexto:

CONTEXTO DE DIAGNÓSTICO:

[COMPORTAMIENTO OBSERVADO]

Desde las 14:35 UTC de hoy, algunos pedidos con descuentos de tipo 'fixed' muestran totales negativos en la UI. El log de pagos muestra que se están procesando como \$0.00 en lugar del valor negativo (hay validación en el servicio de pagos), pero el frontend muestra el valor incorrecto.

[STACK TRACE]

File "orders/services/pricing.py", line 89, in calculate_order_total

```
total = max(Decimal('0'), total - rule.value)
- total: Decimal('-15.00')
- rule.value: Decimal('65.00')
- subtotal antes de descuento: Decimal('50.00')
```

[CÓDIGO DE LAS FUNCIONES IMPLICADAS]

```
# [código completo de calculate_order_total]
# [código de la función que llama a calculate_order_total]
```

[CAMBIOS RECIENTES - git log]

```
a3f91b2 feat: add free shipping promotion (hace 3 horas) - María García
b2e8d41 fix: correct tax rounding in checkout (hace 6 horas) - Juan López
c1d7f30 refactor: extract discount validation (hace 1 día) - Ana Martínez
```

[DIFF DEL COMMIT a3f91b2]

```
# [diff completo del commit de la promoción de envío gratis]
```

[TEST QUE FALLA]

```
FAILED tests/orders/test_pricing.py::test_fixed_discount_with_minimum_amount
AssertionError: assert Decimal('-15.00') == Decimal('35.00')
```

Con este contexto, el modelo puede razonar sobre la causa raíz con precisión: el commit de la promoción de envío gratis (a3f91b2) probablemente modificó el orden en que se aplican las reglas de descuento, o agregó una regla de tipo 'fixed' para el envío gratis con un valor que

supera el subtotal del pedido, produciendo totales negativos para pedidos pequeños que tienen ese descuento.

Sin el contexto del diff reciente y la descripción del comportamiento observado, la misma pregunta produciría solo una explicación genérica del código de la función, que el desarrollador ya conoce.

Debugging de sistemas distribuidos

Los sistemas distribuidos presentan un desafío adicional para el debugging: el error puede manifestarse en un servicio pero originarse en otro. El stack trace de un servicio no revela lo que ocurrió en los servicios upstream que lo llamaron.

El Context Engineering para debugging distribuido agrega al contexto estándar:

- Los logs del servicio upstream en el rango de tiempo del error (con correlation IDs para vincular las trazas)
- El contrato de la API entre los servicios (para verificar si el servicio upstream cumplió su contrato)
- El historial de cambios recientes en los servicios relacionados

Los sistemas de observabilidad modernos (Datadog, Grafana, OpenTelemetry) permiten recuperar este contexto de manera relativamente automatizada cuando están bien configurados. El AI Engineer puede diseñar herramientas que, dado un correlation ID de un error, recuperan automáticamente todos los logs relacionados de los servicios implicados y los ensamblan como contexto para el modelo.

Mantenimiento: contexto para cambios en sistemas legados

El mantenimiento de sistemas existentes — en particular sistemas legados con documentación escasa o desactualizada — es uno de los casos de uso donde el Context Engineering para IA tiene mayor valor potencial y mayor riesgo.

El valor potencial: el modelo puede analizar código que el desarrollador actual no escribió y que no está documentado, y proporcionar un entendimiento de su lógica a partir del código mismo.

El riesgo: el modelo puede generar explicaciones plausibles del comportamiento de código legado que son incorrectas porque la lógica real depende de efectos colaterales, estado global o convenciones no documentadas que el modelo no puede inferir del código solo.

El contexto correcto para trabajo en código legado incluye:

- El código del módulo a modificar, completo
- Cualquier documentación existente, aunque esté desactualizada (la diferencia entre la documentación y el código real es información en sí misma)
- Los tests existentes (si los hay)
- El historial de commits del módulo (puede revelar la intención original)
- Los issues y PRs relacionados con ese módulo (pueden documentar decisiones que nunca llegaron al código)

Y, fundamentalmente, una advertencia explícita al modelo sobre el contexto: "Este es código legado. Puede tener comportamientos no documentados. Las hipótesis que generes sobre su funcionamiento deben considerarse preliminares hasta verificación."

Automatización del contexto de diagnóstico

Para equipos con alta frecuencia de bugs o sistemas complejos, el proceso de construcción del contexto de diagnóstico puede automatizarse parcialmente. Un script que, dado un error en los logs de producción:

1. Recupera el stack trace completo con valores de variables
2. Identifica los archivos de código mencionados en el trace y los recupera del repositorio
3. Ejecuta `git log --oneline -10` en esos archivos para identificar cambios recientes
4. Recupera los tests del módulo afectado
5. Ensambla todo en un prompt de diagnóstico estructurado

Esta automatización no reemplaza el juicio del desarrollador en la evaluación del diagnóstico, pero reduce significativamente el tiempo de preparación del contexto.

Nota del arquitecto

El debugging asistido por IA, bien implementado, reduce el tiempo de diagnóstico. Pero introduce un riesgo sutil: el modelo puede proponer hipótesis de causa raíz convincentes pero incorrectas, especialmente cuando el stack trace y el código disponible son ambiguos. La hipótesis convincente puede llevar al desarrollador a explorar una dirección incorrecta antes de verificarla.

El contrapeso es la verificación explícita antes de actuar: toda hipótesis de causa raíz generada por el modelo debe verificarse con un test o una reproducción local antes de aplicar el fix. El

modelo propone; el desarrollador verifica y decide.

La siguiente sección sale del código en sí mismo para analizar el ecosistema donde ese código vive: los IDEs, repositorios y pipelines de CI/CD, y cómo el Context Engineering opera en esos entornos.

Capítulo 11 — Context Engineering para Desarrollo de Software

Sección 08: Integración con IDEs, repositorios y CI/CD

El Context Engineering no opera en el vacío. Opera dentro del ecosistema de herramientas que los equipos de desarrollo ya usan: IDEs, sistemas de control de versiones, plataformas de CI/CD. Esta sección analiza cómo ese ecosistema determina qué contexto está disponible, cómo se recupera y cómo se entrega al modelo.

El enfoque es deliberadamente agnóstico de herramientas. Las plataformas específicas — GitHub vs. GitLab, GitHub Actions vs. Jenkins, VSCode vs. Cursor — evolucionan rápidamente. Los principios de cómo estructurar el contexto en cada capa del ecosistema permanecen estables.

El IDE como interfaz de contexto

El IDE es el punto de contacto más inmediato entre el desarrollador y el modelo. Las herramientas modernas de asistencia de código (Cursor, GitHub Copilot, Codeium, y similares) operan directamente dentro del IDE y tienen acceso a contexto que otras interfaces no tienen de forma natural.

Contexto disponible en el IDE:

- El archivo actualmente abierto
- Los archivos abiertos en pestañas activas
- El cursor y la selección actual (qué código está mirando el desarrollador)
- El proyecto completo (con limitaciones de tamaño)
- La definición de funciones y tipos accesibles vía Language Server Protocol (LSP)

Lo que el AI Engineer puede configurar:

El contexto disponible en el IDE puede expandirse mediante archivos de instrucciones del proyecto. Herramientas como Cursor permiten definir archivos `.cursorrules` o

`.claude/project.md` que se incluyen automáticamente en el contexto de cada sesión. Estos archivos documentan:

- Las convenciones de código del proyecto
- Los patrones de diseño adoptados
- Las instrucciones específicas para el asistente ("siempre usar Decimal para valores monetarios", "nunca usar print para debugging, usar el logger del proyecto")
- Las restricciones de seguridad ("no incluir credenciales en el código")

Este archivo de instrucciones de proyecto es una de las inversiones de mayor retorno en Context Engineering para IDEs: se escribe una vez y mejora cada sesión de desarrollo en el proyecto.

EJEMPLO: .claude/project.md para un proyecto de e-commerce

```
# Proyecto: E-Commerce Platform Backend

## Stack

- Python 3.11, FastAPI, SQLAlchemy 2.0
- PostgreSQL (producción), SQLite (tests)
- Kafka para mensajería asincrónica
- structlog para logging

## Convenciones de código

- Type hints obligatorios en todas las funciones
- Docstrings en formato Google Style
- Decimal para todos los valores monetarios, nunca float
- Manejo de errores con excepciones custom en each domain

## Patrones de diseño

- Repository pattern para acceso a datos
- Event-driven para comunicación entre dominios
- CQRS para módulos con alta frecuencia de lectura

## Instrucciones para el asistente

- Al generar código de un servicio, verificar que usa el logger del proyecto
- Al generar tests, usar factory_boy para fixtures
- No usar print() para debugging
- Verificar que los imports corresponden a dependencias del pyproject.toml
```

El repositorio como fuente de contexto estructurado

El repositorio de código es la fuente de contexto más rica disponible para el AI Engineer. Contiene no solo el código actual, sino la historia completa de decisiones técnicas.

Los artefactos del repositorio que son fuentes de contexto de alta calidad:

git log con mensajes de commit descriptivos. Los commits con mensajes de la forma "feat(orders): add minimum amount validation for discount rules" son mucho más útiles que

"fix stuff". El mensaje de commit es el contexto del por qué del cambio, que el modelo necesita para razonar sobre el impacto de modificaciones.

Pull requests con comentarios de revisión. Los comentarios en PRs históricos documentan el razonamiento técnico del equipo: por qué una solución fue rechazada, qué trade-offs se evaluaron, qué preocupaciones de seguridad o performance fueron identificadas.

Issues y sus estados. Las issues cerradas documentan qué problemas existieron y cómo se resolvieron. Las issues abiertas documentan el trabajo pendiente y los problemas conocidos.

Archivos de configuración. `pyproject.toml`, `package.json`, `.eslintrc`, `Makefile` — documentan las dependencias, las herramientas de calidad adoptadas y los comandos estándar del proyecto.

El AI Engineer que diseña un sistema de asistencia debe considerar cuáles de estas fuentes se incluyen en el contexto de manera sistemática y cuáles se recuperan bajo demanda.

El pipeline de CI/CD como contexto automatizado

El pipeline de CI/CD es el contexto más dinámico del ecosistema: cambia con cada commit, con cada deployment, con cada ejecución. Y es también el contexto donde la IA puede automatizar tareas de alta frecuencia con mayor impacto.

Los principios para integrar IA en pipelines de CI/CD, independientemente de la plataforma:

Principio 1: El contexto del pipeline debe incluir siempre el diff.

Cualquier tarea que el modelo ejecuta en el pipeline — revisión de código, análisis de seguridad, generación de documentación de release — requiere el diff del cambio que triggera el pipeline. Sin el diff, el modelo no sabe qué cambió.

Principio 2: El contexto del pipeline debe incluir los resultados de pasos anteriores.

Si el pipeline incluye tests unitarios, análisis estático y revisión de código en ese orden, la revisión de código tiene acceso al output de los tests y del análisis estático. Ese output es contexto: informa al modelo sobre problemas ya detectados antes de que realice su propia revisión.

Principio 3: El contexto del pipeline debe respetar las políticas del equipo.

Las políticas de merge del equipo — qué condiciones deben cumplirse para que un PR sea elegible para merge — son contexto que el modelo necesita para evaluar si un cambio cumple los estándares del proyecto. Si la política dice que todo código nuevo debe tener cobertura de tests mayor al 80%, el modelo debe tener acceso a ese requisito para verificar si el PR lo cumple.

ESTRUCTURA DE CONTEXTO EN UN PIPELINE DE CI/CD

[EVENTO]: Pull Request abierto - rama: feat/add-fixed-discount

[DIFF]: [contenido del git diff]

[METADATOS]: Autor, rama base, número de commits, archivos modificados

[PASO 1: Tests automáticos]

Output: ✓ 247 tests pasaron, 0 fallaron

Coverage: 84% (umbral: 80%) → PASA

[PASO 2: Análisis estático]

Output: 2 warnings de tipo - function without return type hint

0 errores críticos → CONTINÚA CON WARNINGS

[PASO 3: Revisión asistida por IA]

Contexto disponible:

- Diff completo
- Output de tests (paso 1)
- Warnings de tipo (paso 2)
- Guías de estilo del proyecto
- Políticas de merge

Tarea: Revisar el PR y comentar sobre:

1. Los warnings de tipo (¿son intencionales o errores?)
2. Casos borde no cubiertos por los nuevos tests
3. Consistencia con las guías del proyecto

Recuperación de contexto del repositorio en tiempo real

Para sistemas de asistencia más sofisticados, el contexto del repositorio se recupera de manera dinámica según la tarea específica. Esto requiere un sistema de indexación del repositorio.

Los elementos que se indexan:

- Código fuente (chunkeado por función o clase)
- Documentación y comentarios
- Mensajes de commit
- ADRs y documentación de arquitectura

La recuperación se realiza mediante similitud semántica (embeddings) o mediante búsqueda léxica (grep avanzado). Cuando el desarrollador formula una tarea, el sistema recupera los fragmentos más relevantes del repositorio y los incluye en el contexto del modelo.

Este enfoque resuelve el problema de repositorios grandes: en lugar de intentar meter todo el codebase en la ventana de contexto (imposible), el sistema recupera selectivamente el contexto más relevante para la tarea específica.

Integración con herramientas de observabilidad

En la fase de mantenimiento y operaciones, las herramientas de observabilidad (sistemas de logging centralizado, métricas, trazas distribuidas) son fuentes de contexto que complementan al repositorio.

El AI Engineer puede diseñar integraciones que, dado un alert de producción, recuperan automáticamente:

- Los logs del servicio en el rango de tiempo del alert
- El stack trace completo del error
- El código del módulo afectado (del repositorio)
- Los commits recientes en ese módulo

Este ensamblaje automático del contexto de diagnóstico — integrando observabilidad y repositorio — reduce el tiempo de respuesta ante incidentes de producción de manera significativa.

Nota del arquitecto

La tentación al diseñar integraciones de IA en el pipeline de CI/CD es hacer la revisión del modelo bloqueante: el pipeline no avanza hasta que el modelo da su aprobación. Esto es un

error de diseño. El modelo puede tener falsos positivos que bloqueen merges legítimos, y puede tener falsos negativos que aprueben código problemático.

El rol correcto de la revisión asistida por IA en el pipeline es de asistencia al humano, no de árbitro automático. El modelo señala problemas; el desarrollador evalúa y decide. El pipeline puede informar sobre los findings del modelo, puede generar comentarios automáticos en el PR, pero la decisión de merge sigue siendo humana.

La siguiente sección sintetiza lo que el capítulo enseña sobre errores frecuentes y patrones de éxito: los anti-patrones que deben evitarse y los patrones que funcionan en producción.

Capítulo 11 — Context Engineering para Desarrollo de Software

Sección 09: Patrones y anti-patrones

Las secciones anteriores describieron cómo construir buen contexto en cada fase del ciclo de vida. Esta sección cataloga los patrones que funcionan de manera consistente y los anti-patrones que generan problemas recurrentes. Estos patrones emergieron de la experiencia práctica de equipos que implementaron Context Engineering en proyectos reales.

Anti-patrones: los errores más costosos

Anti-patrón 1: El Contexto Vacío

El anti-patrón más frecuente y más costoso. El desarrollador formula la petición al modelo sin ningún contexto adicional del proyecto: "Escribe una función que valide un email", "Crea un endpoint REST para pedidos", "Refactoriza esta función". El modelo genera una respuesta genérica que no corresponde al proyecto específico.

El costo no es solo el tiempo de la refactorización posterior. Es la cultura que ese anti-patrón crea: si el equipo experimenta repetidamente que el código generado no sirve, concluye que la IA no es útil para su proyecto. La IA no tenía contexto — no era inútil, era ciega.

Corrección: establecer un archivo de instrucciones del proyecto (`.claude/project.md` o equivalente) que sea incluido automáticamente en cada sesión, y entrenar al equipo en la práctica de proporcionar el módulo y las clases relacionadas antes de cada petición de generación.

Anti-patrón 2: El Copy-Paste Sin Revisión

El desarrollador recibe el código generado por el modelo y lo pega directamente al codebase sin revisión. El código compila. Los tests pasan. Se mergea.

Este anti-patrón es especialmente peligroso porque la mayor parte del tiempo funciona — lo cual genera falsa confianza. Los problemas aparecen en los casos que los tests no cubren: el

código generado asume condiciones que no están garantizadas en el sistema real, viola un invariante de seguridad que estaba documentado en una issue que el modelo no vio, o introduce una dependencia de terceros con una licencia incompatible con el proyecto.

La corrección no es no usar IA — es establecer una cultura de revisión crítica. El código generado por IA debe pasar el mismo escrutinio que el código generado por un desarrollador junior: revisado por un desarrollador con conocimiento del sistema antes de que llegue a main.

Anti-patrón 3: El Contexto Monolítico

El desarrollador intenta resolver este problema yendo al extremo opuesto: incluye todo el repositorio en el contexto de cada petición. Esto degrada la calidad del razonamiento del modelo (la ventana de contexto tiene límites efectivos de atención, no solo de tokens), hace las peticiones lentas y costosas, y mezcla contexto relevante con irrelevante.

El modelo, cuando se le da demasiado contexto irrelevante, tiende a anclar en los primeros elementos del contexto y a dar menos atención a los elementos que aparecen en el medio. Un contexto grande y desorganizado es peor que un contexto selectivo y bien estructurado.

Corrección: contexto selectivo y relevante para la tarea específica. El arte del Context Engineering es elegir qué incluir y qué excluir, no incluir todo.

Anti-patrón 4: La Revisión de IA Como Etapa Final

El equipo configura una revisión automática de IA como el último paso del pipeline antes del merge. Si la IA aprueba, el PR se mergea sin revisión humana adicional.

Este anti-patrón invierte el rol correcto del modelo. La IA debe ser una capa de asistencia para el revisor humano, no un árbitro final. El modelo puede tener falsos negativos — aprobar código con problemas que el contexto disponible no le permitió identificar — y esos falsos negativos llegan a producción sin revisión.

Corrección: la revisión de IA es un paso de asistencia que produce findings para el revisor humano. La aprobación final siempre requiere un humano con conocimiento del sistema.

Anti-patrón 5: El Contexto Desactualizado

El equipo crea un archivo de instrucciones del proyecto y no lo actualiza. El proyecto migra a Python 3.12, adopta un nuevo framework de tests, cambia las convenciones de naming — pero

el archivo de contexto sigue describiendo el estado del proyecto de hace seis meses. El modelo genera código con las convenciones viejas, que el equipo debe corregir manualmente.

Corrección: el archivo de instrucciones del proyecto se trata como documentación viva. Se incluye en el proceso de onboarding de nuevas decisiones técnicas: cuando el equipo adopta una nueva convención, la documenta en el archivo de contexto del proyecto.

Patrones que funcionan

Patrón 1: El Archivo de Contexto del Proyecto

Un archivo mantenido en el repositorio (`.claude/project.md`, `CONTEXT.md`, `.cursorrules` o el equivalente de la herramienta) que documenta:

- Stack tecnológico con versiones
- Convenciones de código
- Patrones de diseño adoptados
- Instrucciones específicas para el asistente

Este archivo se incluye automáticamente en cada sesión de trabajo. Su mantenimiento es responsabilidad del equipo, igual que el CHANGELOG o el README.

Patrón 2: El Contexto Mínimo Suficiente

Para cada tarea, el desarrollador identifica los elementos de contexto sin los cuales el modelo no puede producir un output útil:

- Para generación de código: el módulo destino, las clases del dominio a usar, los tests que debe pasar
- Para debugging: el stack trace, el código de las funciones implicadas, los cambios recientes
- Para revisión de PR: el diff, el requisito que motivó el cambio, las guías de estilo

Nada menos que eso (degradaría la calidad), nada más que eso (desperdiciaría la ventana de contexto).

Patrón 3: Verificación Antes de Integración

El output del modelo siempre pasa por verificación antes de integrarse:

- Código generado: linter → tests unitarios → revisión humana
- Diagnóstico de bug: reproducción local antes de aplicar el fix
- Revisión de PR: evaluación humana de los findings antes de comentar

La IA acelera el trabajo; la verificación asegura la calidad.

Patrón 4: El Contexto Escalonado

Para tareas complejas que requieren múltiples pasos de razonamiento, el contexto se construye de manera escalonada. En lugar de dar todo el contexto de una vez y pedir el resultado final, el desarrollador trabaja en pasos:

1. "Dado este stack trace y este código, ¿qué hipótesis de causa raíz son plausibles?"
2. El desarrollador evalúa las hipótesis y selecciona la más probable.
3. "Dada esta hipótesis de causa raíz, ¿qué fix propondrías?"
4. El desarrollador evalúa el fix propuesto, lo verifica localmente.
5. "Escribe el test que habría detectado este bug antes del deployment."

Cada paso usa el output del paso anterior como contexto adicional, construyendo el razonamiento de manera incremental.

Patrón 5: La Revisión Bidireccional

El desarrollador usa el modelo para revisar su propio trabajo antes de enviarlo a revisión del equipo. Proporciona su código + las guías del proyecto y pide una revisión crítica. Esto captura problemas obvios antes de la revisión de pares, haciendo esa revisión más eficiente.

La revisión bidireccional también tiene un efecto secundario positivo: el desarrollador aprende de los problemas que el modelo señala, lo que mejora su calidad de código a lo largo del tiempo.

TAXONOMÍA DE PATRONES Y ANTI-PATRONES

ANTI-PATRONES:

1. Contexto vacío → Output genérico, no útil para el proyecto
2. Copy-paste sin revisión → Riesgo de bugs y violaciones de política
3. Contexto monolítico → Degradación del razonamiento, costo innecesario
4. IA como árbitro final → Falsos negativos sin contrapeso humano
5. Contexto desactualizado → Output con convenciones obsoletas

PATRONES:

1. Archivo de contexto del proyecto → Contexto base consistente en cada sesión
2. Contexto mínimo suficiente → Calidad máxima con costo mínimo
3. Verificación antes de integrar → Calidad asegurada por proceso
4. Contexto escalonado → Razonamiento incremental de calidad
5. Revisión bidireccional → Mejora antes del proceso formal de revisión

Señales de que el Context Engineering está funcionando

Los equipos que implementan Context Engineering efectivamente observan las siguientes señales:

- El código generado requiere ajustes mínimos antes de ser integrable
- Los desarrolladores junior producen código que sigue las convenciones del proyecto desde el primer intento
- El tiempo de debugging de bugs reportados en producción se reduce
- Los comentarios en PRs generados por IA son específicos y accionables, no genéricos
- Los desarrolladores confían en usar el asistente de IA para tareas complejas, no solo para completar código trivial

Señales de que el Context Engineering está fallando

- El código generado se descarta frecuentemente o requiere reescritura significativa
- El equipo percibe que la IA "no sirve para nuestro proyecto"
- Los comentarios de revisión de IA son genéricos ("considera agregar manejo de errores") sin especificidad del proyecto
- Los bugs introducidos por código generado sin revisión llegan a producción

- El equipo tiene reglas informales como "nunca uses IA para el módulo X" por malas experiencias previas

Estas señales de fallo son casi siempre síntomas de un problema de contexto, no de un problema del modelo.

Nota del arquitecto

La lista de anti-patrones anterior no es exhaustiva, y el orden de impacto varía según el equipo. Pero el anti-patrón 2 (Copy-Paste Sin Revisión) merece énfasis especial porque es el que tiene consecuencias más difíciles de revertir: código con bugs en producción, vulnerabilidades de seguridad introducidas sin intención y deuda técnica acumulada sin visibilidad. La cultura de revisión crítica del código asistido por IA es la base sobre la que todo lo demás funciona.

La siguiente sección materializa estos patrones en un contexto concreto: un caso de estudio empresarial que muestra cómo un equipo real implementó Context Engineering en su ciclo de desarrollo.

Capítulo 11 — Context Engineering para Desarrollo de Software

Sección 10: Caso de estudio empresarial

Esta sección reconstruye la implementación de Context Engineering en un equipo de desarrollo de software de tamaño mediano. El caso es representativo de equipos que han madurado su uso de IA desde herramientas de completado de código hacia sistemas de asistencia integrados en el ciclo de vida completo.

Los nombres del equipo y la organización son ficticios. Los problemas, las decisiones y los resultados son representativos de implementaciones reales.

El contexto inicial: el equipo y sus problemas

El equipo desarrolla el backend de una plataforma de marketplace B2B. Son 12 desarrolladores, organizados en tres squads: Catálogo, Pedidos y Facturación. El stack es Python (FastAPI, SQLAlchemy), PostgreSQL y Kafka. El repositorio tiene aproximadamente 180.000 líneas de código Python.

Antes de la implementación, el equipo ya usaba herramientas de IA de manera informal:

- Varios desarrolladores usaban GitHub Copilot para completado de código
- Algunos usaban Claude o GPT-4 en ventanas separadas para debugging
- El uso era individual e inconsistente: sin convenciones de equipo, sin métricas, sin proceso definido

Los problemas que motivaron una implementación más estructurada:

Inconsistencia del output. El código generado por IA tenía calidad muy variable. Algunos desarrolladores obtenían sugerencias excelentes; otros reportaban que el completado generaba código inutilizable. La diferencia no estaba documentada — cada desarrollador tenía su propia intuición sobre "cómo hablar con la IA".

Revisiones de código más lentas. Los PRs con código asistido por IA frecuentemente incluían violaciones de convenciones del proyecto que alargaban las revisiones. El code style

del proyecto no estaba disponible para el modelo.

Falsa confianza en el debugging. Varios incidentes de producción se investigaron con IA y el diagnóstico inicial fue incorrecto. Los desarrolladores habían proporcionado solo el stack trace, sin el contexto del código ni los cambios recientes.

La intervención: qué se diseñó

El AI Engineer del equipo diseñó una intervención en tres niveles: contexto base del proyecto, flujos de trabajo estandarizados y un sistema de recuperación de contexto del repositorio.

Nivel 1: Contexto base del proyecto

Se creó un archivo `CONTEXT.md` en la raíz del repositorio con las convenciones del equipo, el stack tecnológico, los patrones de diseño adoptados y las instrucciones específicas para el asistente de IA. Este archivo se configuró para ser incluido automáticamente en Cursor (el IDE adoptado como estándar del equipo).

Sección de muestra del archivo:

```
# Backend Platform — Contexto para Asistente IA

## Stack

Python 3.11, FastAPI 0.111, SQLAlchemy 2.0, Alembic, Kafka (confluent-kafka)
PostgreSQL 15 (producción), pytest + SQLite en memoria (tests)

## Convenciones obligatorias

- Decimal para todos los valores monetarios (nunca float)
- structlog para logging (nunca print ni logging stdlib directamente)
- Excepciones custom por dominio: orders/exceptions.py, catalog/exceptions.py
- Type hints obligatorios; Mypy en modo strict
- Tests con factory_boy para fixtures, no con fixtures hardcodeadas

## Patrones del proyecto

- Repository pattern: acceso a base de datos solo a través de repositorios
- Eventos de dominio: cambios de estado se publican a Kafka, nunca se llama
  a otros dominios directamente
- DTOs para entrada/salida de endpoints; modelos SQLAlchemy no se exponen
  directamente en las responses

## Instrucciones para el asistente

- Antes de generar código, verificar que usa las excepciones del dominio
  correcto, no excepciones genéricas de Python
- Los tests de integración usan una base de datos SQLite en memoria;
  no generar tests que requieran conexión a PostgreSQL real
- Todo código nuevo debe tener docstring en formato Google Style
```

Nivel 2: Flujos de trabajo estandarizados

El equipo documentó cuatro flujos de trabajo estándar para los casos de uso más frecuentes:

Flujo de generación de código: el desarrollador abre el módulo destino en Cursor, selecciona las clases relacionadas que usará, describe la función a generar, ejecuta los tests generados.

Flujo de debugging: el desarrollador ejecuta un script `debug-context.sh` que recibe el error de los logs y genera automáticamente un archivo de contexto con el stack trace, el código de

las funciones implicadas, el diff de los últimos 5 commits de esos archivos y los tests que fallan. Ese archivo de contexto se usa como input para el modelo.

Flujo de revisión de PR: el CI/CD ejecuta automáticamente una revisión asistida en cada PR, con el diff, el contexto de las funciones afectadas y las guías del proyecto. Los findings aparecen como comentarios del bot en el PR, para evaluación del revisor humano.

Flujo de generación de tests: el desarrollador proporciona la función a testear + su especificación funcional, y pide la generación de tests. Los tests generados se revisan antes de incluirlos en el PR.

Nivel 3: Sistema de recuperación de contexto del repositorio

Para las tareas más complejas que requieren contexto de múltiples partes del repositorio, el equipo implementó un sistema simple de indexación: un script semanal que genera embeddings de las funciones del repositorio usando sentence-transformers y los almacena en un índice vectorial local (Chroma). Cuando un desarrollador trabaja en una tarea compleja, puede ejecutar `context-retrieval.py "descripción de la tarea"` y el script recupera las 10 funciones más semánticamente similares del repositorio como contexto adicional.

Este sistema no requirió infraestructura en la nube: corre localmente en la máquina del desarrollador y se actualiza semanalmente con el contenido del repositorio.

Los resultados: qué cambió

El equipo midió cuatro métricas antes y después de la implementación, durante un período de tres meses:

Tasa de aceptación del código generado. El porcentaje del código sugerido por IA que se acepta sin modificaciones significativas. Antes: 34%. Después: 61%. La mejora se atribuye principalmente al archivo de contexto del proyecto, que evita las violaciones de convenciones más frecuentes.

Tiempo promedio de revisión de PR. El tiempo desde que se abre un PR hasta que recibe su primera revisión humana. Antes: 4.2 horas. Después: 2.8 horas. La revisión automática de IA identifica los problemas más obvios antes de que el revisor humano abra el PR, haciendo su revisión más focada.

Tiempo de diagnóstico de incidentes. El tiempo desde que se reporta un error de producción hasta que el equipo tiene una hipótesis de causa raíz. Antes: 47 minutos (promedio de 12 incidentes del período previo). Después: 18 minutos (promedio de 9

incidentes del período posterior). La reducción se atribuye principalmente al script `debug-context.sh` y al flujo estandarizado de debugging.

Satisfacción del equipo con las herramientas de IA. Survey interno. Antes: 3.1/5 ("útil para tareas simples"). Después: 4.2/5 ("útil para tareas complejas también"). El cambio más significativo fue en la percepción de los desarrolladores senior, que antes evitaban las herramientas de IA para tareas complejas.

Las lecciones del caso

La documentación del proyecto ya existía parcialmente. El equipo tenía convenciones documentadas en un wiki que nadie leía. El paso más impactante fue hacer esa documentación accesible al modelo en el formato correcto (archivo en el repositorio, no en el wiki). El contenido ya existía; el problema era la accesibilidad.

El cambio cultural tomó más tiempo que el técnico. Implementar el archivo de contexto y el script de debugging tomó dos semanas. Lograr que todo el equipo lo usara de manera consistente tomó tres meses. La resistencia no era técnica — era de hábito. Los desarrolladores con años de uso individual de herramientas de IA tenían sus propias rutinas y les costaba adoptar el flujo estandarizado.

Los flujos de trabajo fallaron antes de funcionar bien. El primer intento del flujo de revisión de PR generó demasiados falsos positivos — el modelo señalaba como problemas estilos de código perfectamente válidos que no estaban documentados en el archivo de contexto. El equipo refinó el archivo de contexto durante dos semanas de feedback hasta que la tasa de falsos positivos bajó a un nivel aceptable.

El tamaño del contexto importa. El equipo probó inicialmente incluir el repositorio completo en el contexto de las revisiones de PR. La calidad de los findings cayó y el costo de las llamadas al modelo aumentó. La reducción al contexto mínimo suficiente (diff + funciones afectadas + guías del proyecto) mejoró ambas métricas.

Nota del arquitecto

Este caso ilustra un patrón general de implementación: el valor de Context Engineering no está en la complejidad técnica de la solución — el archivo de contexto del proyecto es un archivo Markdown, el script de debugging es un script de shell — sino en la consistencia de su aplicación. El equipo que aplica principios simples de manera consistente obtiene mejores resultados que el equipo que implementa sistemas sofisticados de manera inconsistente.

La siguiente sección proporciona al lector la oportunidad de aplicar estos principios directamente: el laboratorio práctico del capítulo.

Capítulo 11 — Context Engineering para Desarrollo de Software

Sección 11: Laboratorio práctico

Este laboratorio tiene un objetivo preciso: demostrar, de manera ejecutable, que el contexto determina la calidad del diagnóstico asistido por IA. El ejercicio trabaja con un repositorio de ejemplo que contiene un bug real, y el estudiante construye el contexto mínimo necesario para que el modelo identifique correctamente el problema.

La elección del debugging como ejercicio del laboratorio es deliberada: es el caso de uso donde el impacto del contexto es más dramático y más fácil de medir objetivamente. Un diagnóstico correcto vs. uno incorrecto es verificable sin ambigüedad.

Descripción del escenario

El repositorio de práctica contiene el backend simplificado de un sistema de procesamiento de pedidos. El sistema tiene el siguiente bug en producción:

Pedidos con múltiples descuentos aplicados muestran totales incorrectos cuando el segundo descuento tiene un monto mínimo mayor al subtotal después de aplicar el primer descuento.

El sistema recibe el bug como un reporte de un cliente:

REPORTE: ORD-2024-0847

Fecha: hoy, 09:23 UTC

Descripción: El pedido ORD-2024-0847 muestra un total de \$89.10 en la confirmación de compra, pero el cliente recibió un cargo de \$99.00 en su tarjeta. Los descuentos aplicados: 10% por volumen + \$5 de descuento por primera compra.

El test suite también reporta un fallo:

```
FAILED tests/orders/test_pricing.py::test_stacked_discounts_with_minimum
AssertionError:

Expected: Decimal('89.10')

Got: Decimal('99.00')

Input: order.subtotal = Decimal('110.00')

       discount_1: type='percentage', value=10    [sin mínimo]
       discount_2: type='fixed', value=Decimal('10.90') [mínimo: Decimal('95.00')]
```

El ejercicio: construir el contexto mínimo

El objetivo del ejercicio no es encontrar el bug — es construir el contexto correcto para que el modelo lo encuentre. Esto requiere que el estudiante decida:

1. ¿Qué información del repositorio es relevante para este diagnóstico?
2. ¿Cómo estructurar esa información en el prompt?
3. ¿Qué está omitiendo que impide que el modelo llegue a la causa raíz?

Paso 1: El intento sin contexto

El estudiante formula el primer prompt solo con el reporte del cliente:

```
PROMPT (sin contexto):

Un cliente reporta que su pedido ORD-2024-0847 muestra un total de $89.10
en la confirmación pero recibió un cargo de $99.00.

¿Cuál puede ser la causa?
```

El modelo responde con hipótesis genéricas: descuento no aplicado correctamente, error de redondeo, problema de sincronización entre el total de la confirmación y el cargo procesado, etc. Ninguna es específica al código del proyecto.

Paso 2: Agregar el stack trace y el test fallido

```
PROMPT (con stack trace y test):  
  
El sistema de procesamiento de pedidos tiene el siguiente test fallido:  
  
[FALLO DE TEST]  
  
FAILED test_stacked_discounts_with_minimum  
  
Expected: Decimal('89.10'), Got: Decimal('99.00')  
  
Input: subtotal=$110.00, discount_1=10% sin mínimo,  
       discount_2=$10.90 fijo con mínimo $95.00  
  
¿Cuál puede ser la causa?
```

El modelo puede inferir que el segundo descuento no se está aplicando — el total de \$99.00 corresponde al subtotal después del primer descuento (10% de \$110 = \$99), sin el segundo. Pero no puede saber por qué: si es un bug de lógica de condición, un bug de orden de operaciones, o un bug de comparación de tipos.

Paso 3: Agregar el código de la función

```
# CÓDIGO DE calculate_order_total (versión con el bug):

def calculate_order_total(
    order: Order,
    discount_rules: list[DiscountRule],
) -> Decimal:
    subtotal = sum(item.unit_price * item.quantity for item in order.items)
    total = subtotal

    for rule in discount_rules:
        if rule.min_order_amount and total < rule.min_order_amount:
            continue

        if rule.type == 'percentage':
            total -= total * (rule.value / Decimal('100'))
        elif rule.type == 'fixed':
            total = max(Decimal('0'), total - rule.value)

    return total
```

Con el código, el modelo puede identificar el bug: la condición `total < rule.min_order_amount` compara el total **después** de aplicar los descuentos anteriores, no el subtotal original. El segundo descuento tiene un mínimo de \$95.00, y el total después del primer descuento es \$99.00. La condición `$99 < $95` es False, así que el descuento se aplica. Espera — eso significa que sí se aplica. El modelo necesita verificar con los valores exactos.

Paso 4: Agregar el historial de cambios recientes

```
GIT LOG (últimos 5 commits en orders/services/pricing.py):

a3f91b2 feat: evaluate discounts against running total, not subtotal
b2e8d41 fix: correct decimal precision in percentage calculation
c1d7f30 refactor: extract discount rule validation
d8e9a10 test: add integration tests for stacked discounts
e5b2f1a feat: add minimum order amount support for discount rules
```

El commit `a3f91b2` ("evaluate discounts against running total, not subtotal") es el cambio que introdujo el bug. El modelo puede ahora formular la hipótesis precisa: el cambio modificó la variable comparada en la condición del mínimo, de `subtotal` a `total` (el total acumulado después de descuentos anteriores). Esto puede hacer que un descuento que debería aplicar (el total original supera el mínimo) no aplique (el total reducido ya no supera el mínimo), o viceversa.

En el caso del reporte: el total después del 10% de descuento es \$99.00, que supera el mínimo de \$95.00. El descuento debería aplicarse. Pero el test dice que no se aplica. La hipótesis ahora es: ¿la comparación usa `<` cuando debería usar `<=` ? ¿O hay un problema de tipo entre Decimal?

Paso 5: Verificar la hipótesis

El estudiante revisa el diff del commit `a3f91b2`:

```
- if rule.min_order_amount and subtotal < rule.min_order_amount:
+ if rule.min_order_amount and total < rule.min_order_amount:
```

El cambio es exactamente lo que el modelo predijo. Pero la hipótesis del bug es: el total después del primer descuento (10% de \$110 = \$99.00) es mayor que el mínimo (\$95.00), entonces la condición `$99 < $95` es False, y el descuento debería aplicarse.

El estudiante ejecuta el test localmente con el código del commit anterior (revertiendo el cambio) y confirma que el test pasa. El bug es el cambio de `subtotal` a `total` en la condición del mínimo — no porque el cálculo sea incorrecto conceptualmente (evaluar contra el total corriente puede ser un comportamiento deseado en algunos sistemas), sino porque el cambio rompió el contrato documentado en los tests existentes y en el comportamiento que el cliente esperaba.

La lección del ejercicio

El diagnóstico del bug requirió cuatro capas de contexto para ser completo y verificable:

Contexto	Lo que agregó al diagnóstico
Reporte del cliente + test fallido	Síntoma y valores exactos del fallo
Código de la función	Candidatos de mecanismos del bug
Git log reciente	Identificación del commit que introdujo el cambio
Diff del commit	Verificación de la hipótesis exacta

Sin ninguna de estas capas, el diagnóstico hubiera sido incompleto. Con cada capa adicional, el diagnóstico se volvió más preciso y más verificable.

Extensión del laboratorio: el test de regresión

Como extensión, el estudiante escribe el test que habría detectado este bug antes del deployment:

```
def test_stacked_discounts_uses_subtotal_for_minimum_check():
    """
    Verifica que el mínimo de un descuento se evalúa contra el subtotal
    original, no contra el total reducido por descuentos anteriores.

    Regresión para: commit a3f91b2, bug reportado en ORD-2024-0847
    """
    items = [OrderItemFactory(unit_price=Decimal('110.00'), quantity=1)]
    order = OrderFactory(items=items)

    # Primer descuento: 10% sin mínimo → total $99.00
    # Segundo descuento: $10.90 fijo con mínimo $95.00
    # El subtotal original ($110) supera el mínimo, por lo que debe aplicar.
    discount_rules = [
        DiscountRule(type='percentage', value=Decimal('10')),
        DiscountRule(
            type='fixed',
            value=Decimal('10.90'),
            min_order_amount=Decimal('95.00'),
        ),
    ]

    result = calculate_order_total(order, discount_rules)

    # $110 - 10% = $99.00; $99.00 - $10.90 = $88.10
    assert result == Decimal('88.10'), (
        f"Expected $88.10 but got ${result}. "
        "Check that min_order_amount is evaluated against original subtotal."
    )
```

Este test, agregado al commit de fix, habría prevenido regresiones futuras del mismo bug.

Nota del arquitecto

El laboratorio ilustra un principio general: la construcción del contexto de diagnóstico no es una operación de un solo paso. Es un proceso iterativo donde cada capa de contexto agregada

revela la capa siguiente que se necesita. El AI Engineer que diseña sistemas de debugging asistido debe diseñar flujos que faciliten esta construcción incremental, no solo flujos que entreguen todo el contexto posible de una vez.

La siguiente sección consolida las prácticas del capítulo en un checklist accionable para el AI Engineer.

Capítulo 11 — Context Engineering para Desarrollo de Software

Sección 12: Checklist del AI Engineer

Este checklist está diseñado para uso práctico: puede imprimirse, adaptarse al contexto específico del proyecto e incorporarse a los procesos del equipo. Cubre las decisiones de Context Engineering en cada fase del ciclo de vida del software.

Checklist de configuración inicial del proyecto

Antes de comenzar a usar asistencia de IA en un proyecto de software, verificar:

Documentación base del proyecto

- [] Existe un archivo de instrucciones del proyecto (`.claude/project.md` , `CONTEXT.md` o equivalente) en el repositorio
- [] El archivo documenta el stack tecnológico con versiones específicas
- [] El archivo documenta las convenciones de código del equipo
- [] El archivo documenta los patrones de diseño adoptados
- [] El archivo incluye instrucciones específicas para el asistente (qué hacer y qué no hacer)
- [] El archivo está configurado para ser incluido automáticamente en el IDE del equipo
- [] Existe un proceso para mantener el archivo actualizado cuando cambian las convenciones

Configuración del repositorio como fuente de contexto

- [] Los mensajes de commit del equipo siguen una convención descriptiva (no mensajes como "fix" o "changes")
- [] Los ADRs del proyecto están disponibles en el repositorio (no solo en wikis externos)
- [] Existe documentación de las convenciones de testing (frameworks, estructura de tests, uso de mocks)
- [] El README del proyecto describe la estructura del repositorio de manera que el modelo pueda orientarse

Preparación del pipeline de CI/CD

- [] El pipeline proporciona contexto suficiente para las tareas de IA que ejecuta (diff,

resultados de tests previos, políticas del equipo)

- [] Las revisiones automáticas de IA generan comentarios en el PR, no decisiones de merge automáticas
- [] Existe un proceso para evaluar los findings de la revisión automática

Checklist por fase del ciclo de vida

Análisis y relevamiento

- [] El contexto de la sesión incluye las transcripciones o documentos fuente relevantes (no solo las preguntas)
- [] Se identificaron explícitamente qué documentos son de análisis de la tarea actual vs. documentos de otro dominio
- [] Los outputs del análisis asistido (especificaciones, user stories) fueron validados con stakeholders antes de cerrar la fase
- [] Los artefactos de análisis están en formato que las fases posteriores puedan usar como contexto

Diseño y arquitectura

- [] El contexto de las sesiones de diseño incluye los ADRs relevantes del proyecto
- [] Los principios arquitectónicos establecidos del proyecto están en el contexto antes de pedir evaluación de alternativas
- [] Las restricciones no funcionales (latencia, throughput, presupuesto) están documentadas y accesibles al modelo
- [] Los borradores de ADRs generados por IA fueron revisados por el arquitecto antes de ser aprobados
- [] Se verificó que el diseño propuesto es consistente con los requisitos funcionales del análisis

Generación de código

- [] El módulo destino del código está abierto/seleccionado en el IDE
- [] Las clases del dominio que el código debe usar están en el contexto
- [] Los tests que el código debe pasar están en el contexto (o se generaron como primer paso)
- [] Las convenciones del proyecto están en el contexto (via archivo de instrucciones o selección manual)
- [] El código generado pasó el linter antes de ser evaluado por el desarrollador
- [] El código generado tiene al menos una revisión humana antes de incluirse en el PR

Pruebas y QA

- [] Los tests se generan con la especificación funcional en el contexto, no solo el código
- [] Los tests existentes del módulo están en el contexto para evitar duplicación
- [] Los tests generados verifican comportamiento contra especificación (no solo que el código no lanza excepciones)
- [] Los casos borde identificados por el modelo fueron evaluados manualmente para determinar cuáles son relevantes
- [] El contexto de revisión de PR incluye el diff + funciones afectadas + requisito motivador + guías del proyecto

Depuración

- [] El contexto de diagnóstico incluye: comportamiento observado vs. esperado (no solo el mensaje de error)
- [] El stack trace completo está en el contexto
- [] El código de las funciones que aparecen en el stack trace está en el contexto
- [] El git log reciente de los archivos afectados está en el contexto
- [] Los tests que fallan están en el contexto
- [] Las hipótesis de causa raíz generadas por el modelo fueron verificadas localmente antes de aplicar fixes
- [] Se escribió un test de regresión para el bug antes de cerrar el fix

Integración y despliegue

- [] El pipeline incluye el diff del cambio como contexto para cualquier tarea de IA
- [] Los resultados de pasos anteriores del pipeline están disponibles como contexto para pasos posteriores
- [] Las políticas de merge del equipo están documentadas y accesibles al modelo
- [] Los logs de fallo del pipeline incluyen suficiente contexto para diagnóstico asistido

Checklist de calidad del sistema de asistencia

Para evaluar periódicamente si el sistema de Context Engineering está funcionando:

Señales de salud

- [] La tasa de aceptación del código generado sin modificaciones significativas es creciente
- [] Los comentarios de revisión de IA son específicos al proyecto (no genéricos)
- [] El tiempo de diagnóstico de incidentes se redujo respecto al baseline
- [] El equipo usa las herramientas de IA para tareas complejas, no solo para completado trivial

- [] Los desarrolladores nuevos en el proyecto usan el asistente productivamente desde las primeras semanas

Señales de alerta

- [] Los desarrolladores reportan que el código generado "no sirve para nuestro proyecto"
- [] Hay reglas informales de "no usar IA para el módulo X"
- [] Los falsos positivos de la revisión automática superan el 20% de los comentarios
- [] El archivo de instrucciones del proyecto no ha sido actualizado en más de 3 meses
- [] El código de producción contiene bugs cuya causa raíz fue un fix de IA no revisado

Acciones de mantenimiento

- [] El archivo de instrucciones del proyecto se revisa cuando se adoptan nuevas convenciones
- [] El equipo tiene una retrospectiva periódica sobre el uso de herramientas de IA (mensual o por sprint)
- [] Las métricas de uso y calidad del sistema se miden y comparten con el equipo

Preguntas de diagnóstico para el AI Engineer

Cuando el sistema de asistencia no está produciendo los resultados esperados, estas preguntas ayudan a identificar el problema:

1. ¿El modelo tiene acceso a las convenciones del proyecto? Si el código generado viola las convenciones, probablemente no las tiene.
2. ¿El modelo tiene acceso a las clases del dominio que debe usar? Si el código generado inventa sus propias estructuras de datos, probablemente no las tiene.
3. ¿El modelo tiene acceso a las restricciones de la tarea? Si el código generado ignora casos borde documentados en la especificación, esa especificación no está en el contexto.
4. ¿El modelo tiene demasiado contexto irrelevante? Si las respuestas son lentas, genéricas o parecen no leer el contexto específico de la pregunta, puede haber exceso de contexto.
5. ¿El flujo de revisión está funcionando? Si los bugs de IA llegan a producción, el problema puede no ser el modelo sino la ausencia de revisión antes de la integración.

Nota del arquitecto

Este checklist está diseñado para ser adaptado, no seguido mecánicamente. Cada proyecto tiene un contexto diferente: lo que es crítico en un equipo de 5 personas que trabaja en un microservicio puede ser excesivo para un equipo de 30 que trabaja en un monolito complejo.

El AI Engineer debe identificar qué puntos del checklist tienen mayor impacto en su contexto específico y comenzar por ahí.

La siguiente sección resume las ideas centrales del capítulo para consolidar el aprendizaje.

Capítulo 11 — Context Engineering para Desarrollo de Software

Sección 13: Resumen del capítulo

Este capítulo estudió el Context Engineering aplicado al ciclo de vida del desarrollo de software. El recorrido comenzó con una distinción fundamental y terminó con patrones concretos, un caso de estudio y un laboratorio práctico. Esta sección consolida las ideas centrales.

La idea central

El Context Engineering para desarrollo de software parte de una observación: la calidad de la asistencia que la IA puede ofrecer en cualquier tarea de software es una función directa de la calidad del contexto disponible. El mismo modelo, con el mismo prompt, produce resultados radicalmente diferentes según qué información tenga en el contexto.

Esta no es una observación teórica. El experimento de la sección 05 lo demostró con código concreto: la misma petición de generación de código, sin contexto, produce código genérico no integrable; con contexto del módulo, las clases del dominio, las convenciones del proyecto y los tests existentes, produce código directamente integrable. La diferencia es el contexto, no el modelo.

El ciclo de vida como organizador del problema

El capítulo organizó el problema de Context Engineering siguiendo las fases del ciclo de vida del software porque cada fase plantea un problema de contexto diferente.

Análisis y relevamiento trabaja con información heterogénea y no estructurada: transcripciones, documentos de negocio, sistemas legados. El contexto debe seleccionar el material relevante para la tarea de análisis específica.

Diseño y arquitectura trabaja con restricciones técnicas, principios establecidos y trade-offs de largo plazo. El contexto debe incluir los ADRs relevantes, las restricciones no

funcionales y los principios arquitectónicos del proyecto. El modelo amplifica el razonamiento del arquitecto; no lo reemplaza.

Generación de código es la fase de mayor impacto inmediato. El contexto mínimo suficiente incluye: el módulo destino, las clases del dominio a usar, las convenciones del proyecto, las funciones relacionadas existentes y los tests que el código debe pasar.

Pruebas y QA se beneficia del Context Engineering en dos dimensiones: la generación de tests más completos (con la especificación en el contexto, no solo el código) y la revisión de pull requests más específica (con el diff, el requisito motivador y las guías del proyecto).

Depuración y mantenimiento tiene el mayor impacto de tiempo ahorrado. El contexto de diagnóstico correcto incluye el comportamiento observado, el stack trace completo, el código de las funciones implicadas y el historial de cambios recientes. Sin estas capas, el diagnóstico asistido es superficial; con ellas, puede ser preciso.

Integración y CI/CD requiere que el pipeline proporcione el diff, los resultados de pasos anteriores y las políticas del equipo como contexto para las tareas de IA que ejecuta.

Los principios transversales

Cuatro principios se aplican en todas las fases:

Contexto mínimo suficiente. No incluir todo el repositorio — incluir lo que es necesario para la tarea específica. La selección es una habilidad, no un paso técnico.

Verificación antes de integración. El output de la IA siempre pasa por verificación antes de integrarse al proyecto: tests, linter, revisión humana. La velocidad de la IA no reemplaza el rigor de la verificación.

El modelo amplifica, el humano decide. El modelo asiste en la síntesis de información, la generación de opciones y la identificación de problemas. Las decisiones de diseño, de arquitectura y de qué código va a producción son responsabilidad del profesional.

El contexto base del proyecto es la inversión más rentable. Un archivo de instrucciones del proyecto bien mantenido mejora cada sesión de trabajo de cada miembro del equipo, sin costo adicional por sesión.

El anti-patrón más crítico

De todos los anti-patrones identificados en la sección 09, el más importante para el lector recordar es el anti-patrón del copy-paste sin revisión: recibir código generado por IA e

integrarlo directamente al codebase sin revisión humana.

El riesgo no está en que el modelo sea malo — es en que el modelo produce código plausible que puede ser incorrecto, que puede violar convenciones del proyecto, que puede introducir vulnerabilidades de seguridad o que puede violar invariantes de negocio que no estaban en el contexto. La detección de estos problemas requiere revisión humana, no solo ejecución de tests.

La cultura de revisión crítica del código asistido por IA es la base sobre la que todo lo demás funciona.

El ecosistema de herramientas

El Context Engineering no opera aisladamente de las herramientas del equipo. Operan en un ecosistema donde el IDE, el repositorio y el pipeline de CI/CD son fuentes naturales de contexto que ya existen.

El AI Engineer que diseña sistemas de asistencia al desarrollo aprovecha ese ecosistema: el archivo de instrucciones del proyecto vive en el repositorio; el sistema de recuperación de contexto indexa el codebase; el pipeline incluye el diff y los resultados de pasos anteriores como contexto. No se crea infraestructura de contexto desde cero — se conectan las fuentes que ya existen.

Lo que el capítulo no es

El capítulo no es un tutorial de herramientas de IA para programadores. No describió cómo configurar GitHub Copilot, no comparó Cursor con VSCode y no cubrió cómo instalar extensiones de IA en un IDE específico. Esas herramientas cambian y sus documentaciones son la fuente correcta para ese nivel de detalle.

Lo que el capítulo sí es: un marco conceptual y un conjunto de patrones prácticos para diseñar el contexto correcto en cada fase del ciclo de vida del software. Ese marco es estable porque no depende de una herramienta específica.

Conexión con el módulo

El capítulo 11 aplica, en el dominio del desarrollo de software, los principios que el Módulo 3 estableció sobre Context Engineering. Los mecanismos de recuperación de contexto del repositorio son instancias del RAG estudiado en capítulos anteriores. Las instrucciones del proyecto son una forma de memoria externa del sistema. La gestión del contexto a través del ciclo de vida es una forma de memoria episódica del proceso de desarrollo.

La siguiente sección permite al lector evaluar su comprensión de estos conceptos antes de avanzar al capítulo 12.

Capítulo 11 — Context Engineering para Desarrollo de Software

Sección 14: Autoevaluación

Las siguientes preguntas permiten verificar la comprensión de los conceptos centrales del capítulo. Las respuestas no son de memorización sino de aplicación: cada pregunta plantea un escenario y requiere razonar a partir de los principios estudiados.

1. Un desarrollador pega el siguiente prompt en su asistente de IA:

"Escribe una función en Python que valide un número de tarjeta de crédito usando el algoritmo de Luhn."

El modelo genera una función correcta que implementa el algoritmo de Luhn. El desarrollador la integra directamente al módulo de pagos del proyecto. ¿Qué problemas puede haber en el código integrado, y qué contexto habría reducido esos problemas?

2. Un equipo tiene el siguiente workflow: el pipeline de CI/CD ejecuta una revisión automática de IA de cada PR. Si la IA no encuentra problemas, el PR se mergea automáticamente sin revisión humana. Si la IA encuentra problemas, un humano revisa.

Identifica los problemas de diseño de este workflow y propón un diseño alternativo.

3. Un AI Engineer diseña un sistema de debugging asistido para un equipo. El sistema recupera automáticamente el error de los logs de producción y se lo envía al modelo como prompt. Los desarrolladores reportan que el diagnóstico es "casi siempre superficial e inútil". Sin cambiar el modelo, ¿qué haría el AI Engineer para mejorar el sistema?

4. Un equipo tiene el siguiente archivo de instrucciones del proyecto:

```
# Proyecto Backend  
  
- Usamos Python  
  
- El código debe ser limpio y legible  
  
- Seguimos las mejores prácticas de programación
```

Evalúa la calidad de este archivo como contexto y reescríbelo para un proyecto de e-commerce ficticio con las características que consideres representativas.

5. Un desarrollador tiene la especificación de una función: "Calcular el precio de envío de un pedido según el peso y la zona de destino, aplicando tarifas de una tabla de precios del courier." El desarrollador quiere usar IA para generar la función. Lista en orden los elementos de contexto que incluirías en el prompt, justificando por qué cada uno es necesario.

6. Un equipo con 8 desarrolladores quiere implementar Context Engineering en su proyecto. El proyecto tiene 5 años de antigüedad, 220.000 líneas de código y documentación desactualizada. El AI Engineer tiene presupuesto para dedicar 2 semanas a la implementación inicial. ¿Qué haría en esas 2 semanas para maximizar el impacto?

7. Diferencia, con un ejemplo concreto para cada caso, entre:

- Un uso de IA en diseño de arquitectura donde el modelo amplifica el trabajo del arquitecto
- Un uso de IA en diseño de arquitectura donde el modelo está siendo usado como árbitro de la decisión

¿Cuál es el problema del segundo caso?

8. El laboratorio de la sección 11 demostró que el diagnóstico de un bug se volvió más preciso a medida que se agregaban capas de contexto. ¿En qué orden se deben agregar esas capas, y por qué ese orden específico?

9. Un equipo reporta la siguiente experiencia: "Cuando usamos la IA para generar tests de una función nueva, los tests siempre pasan. Pero cuando hay bugs en producción, los tests no los detectan." ¿Cuál es la causa más probable de este problema, y cómo lo corregirías desde el Context Engineering?

10. ¿Por qué el capítulo recomienda que las revisiones automáticas de IA en el pipeline de CI/CD sean informativas (generan comentarios) y no bloqueantes (detienen el pipeline hasta aprobación del modelo)?

Respuestas orientativas

1. La función puede ser correcta en abstracto pero incorrectamente integrada al proyecto: usa tipos de datos incorrectos (str en lugar del tipo personalizado CardNumber del proyecto), no usa las excepciones del dominio de pagos, no incluye logging, no sigue las convenciones de naming del proyecto. El contexto que habría reducido estos problemas: el módulo de pagos existente, las clases del dominio de pagos, las excepciones custom, el archivo de instrucciones del proyecto.

2. Problemas: falsos negativos del modelo (no detecta problemas que existen) llevan a merges sin revisión; el modelo no tiene suficiente contexto del sistema para detectar todos los problemas. Diseño alternativo: la revisión de IA genera comentarios en el PR para el revisor humano; la aprobación final siempre requiere un humano con conocimiento del sistema; el modelo es una capa de asistencia al revisor, no un árbitro.

3. El sistema envía solo el error — el contexto mínimo. El AI Engineer agregaría: el stack trace completo, el código de las funciones del stack trace, el historial de commits recientes de esos archivos, los tests que fallan. El sistema podría automatizar este ensamblaje mediante un script que, dado el error de los logs, recupera automáticamente esos elementos del repositorio.

4. El archivo original es demasiado genérico para ser útil al modelo. Un archivo efectivo especifica el stack con versiones, las convenciones concretas (no "código limpio" sino "type hints obligatorios, Decimal para valores monetarios"), los patrones de diseño adoptados y las instrucciones específicas para el asistente.

5. En orden: (1) el módulo donde se insertará el código — orienta al modelo sobre el dominio; (2) la tabla de precios del courier y cómo está representada en el sistema — el modelo necesita la estructura de datos real; (3) la clase Order o los parámetros de entrada — el modelo necesita los tipos que recibirá la función; (4) funciones similares existentes en el módulo — el modelo puede seguir el mismo patrón; (5) los tests que debe pasar — si existen; (6) las convenciones del proyecto — vía archivo de instrucciones.

6. La inversión de mayor impacto con 2 semanas: (1) Crear el archivo de instrucciones del proyecto a partir de la documentación existente, el código y entrevistas breves con los desarrolladores sénior (1 semana); (2) Estandarizar los flujos de trabajo de debugging y revisión de PR, con el archivo de instrucciones como base (3 días); (3) Capacitar al equipo con un workshop práctico sobre cómo usar el contexto correctamente (2 días). No es recomendable invertir esas 2 semanas en infraestructura técnica compleja (sistemas de RAG, indexación avanzada) hasta que los fundamentos estén funcionando.

7. Amplificación: el arquitecto propone un diseño y le pide al modelo que identifique riesgos no considerados y alternativas que no evaluó. El arquitecto usa ese output para revisar su decisión. Árbitro: el arquitecto presenta dos alternativas y le pregunta al modelo "¿cuál es mejor?", y adopta la respuesta como la decisión. El problema del segundo caso: el modelo puede no tener el contexto organizacional, las capacidades del equipo y las restricciones tácitas que hacen que una alternativa sea superior en ese proyecto específico.

8. El orden correcto es el que agrega valor diagnóstico con cada capa: primero el síntoma (reporte + test fallido), que define qué se está buscando; después el código, que revela el mecanismo potencial; después el historial de cambios, que revela cuándo se introdujo el problema; después el diff específico, que permite verificar la hipótesis. El orden importa porque cada capa informa qué buscar en la siguiente.

9. La causa más probable: los tests se generaron con solo el código en el contexto, sin la especificación funcional. Los tests verifican el comportamiento del código actual (que puede tener bugs) en lugar de verificar el comportamiento esperado según la especificación. La corrección: generar tests con la especificación funcional en el contexto, no solo el código. Los tests deben describir el comportamiento esperado; si el código tiene bugs, los tests los detectarán porque difieren de la especificación.

10. El modelo puede tener falsos negativos — aprobar código con problemas porque el contexto disponible no incluía la información necesaria para detectarlos. Si la revisión es bloqueante y el modelo aprueba, el código llega a producción sin revisión. Si la revisión es

informativa, el humano puede identificar problemas que el modelo no detectó. La revisión automática es una capa de asistencia adicional al proceso de revisión, no un reemplazo de él.

La siguiente y última sección del capítulo establece el puente hacia el capítulo 12 y el territorio que espera al lector.

Capítulo 11 — Context Engineering para Desarrollo de Software

Sección 15: Transición al Capítulo 12

El capítulo 11 estudió el Context Engineering aplicado a un dominio específico: el ciclo de vida del desarrollo de software. Fue el primer capítulo de aplicación del Módulo 3 — el momento en que los principios abstractos del Context Engineering encuentran un territorio concreto y verificable.

Este recorrido no fue arbitrario. El desarrollo de software es el dominio de trabajo central del AI Engineer y del Arquitecto de IA. Antes de aplicar Context Engineering a procesos de negocio más amplios, tiene sentido entenderlo en el territorio más familiar: el propio trabajo de construcción del software.

Lo que el capítulo 11 demostró

El capítulo demostró tres cosas que el lector puede llevar como principios operativos al capítulo siguiente:

Primero: el contexto doméstico es la mayor fuente de valor. Los artefactos del ciclo de vida del software — código, commits, issues, ADRs, tests, especificaciones — son contexto de alta calidad que ya existe. El problema no es crearlo sino conectarlo al sistema de asistencia de manera selectiva y relevante. Esta observación se generaliza al capítulo 12: en cualquier proceso de negocio, existen artefactos que son contexto de alta calidad. El AI Engineer los identifica y los conecta.

Segundo: cada tarea tiene un perfil de contexto diferente. No existe un único contexto correcto para "usar IA en software". El contexto para generar código es diferente al contexto para depurar un bug, que es diferente al contexto para revisar un PR, que es diferente al contexto para diseñar una arquitectura. Esta especificidad del contexto por tarea se generaliza al capítulo 12: cada proceso de negocio tiene un conjunto de tareas con perfiles de contexto propios.

Tercero: la calidad del proceso humano determina la calidad del contexto. Los equipos con buenas prácticas de documentación, mensajes de commit descriptivos y ADRs mantenidos tienen contexto de mejor calidad disponible para la IA. Los equipos con prácticas débiles tienen contexto pobre, independientemente de qué modelo usen. En el capítulo 12, este principio aparece como: la madurez de los procesos de negocio de una organización determina la calidad del contexto que sus sistemas de IA pueden aprovechar.

La escala que cambia en el capítulo 12

El capítulo 12 extiende el Context Engineering al dominio empresarial: procesos de negocio que van más allá del ciclo de vida del software.

El ciclo de vida del software es un dominio relativamente acotado: tiene actores definidos (desarrolladores, arquitectos, testers), artefactos con estructura conocida (código, tests, PRs, ADRs) y criterios de corrección verificables (el código compila, los tests pasan, el requisito se cumple).

Los procesos de negocio empresariales son más amplios en varias dimensiones:

Más actores. El proceso de aprobación de un crédito involucra al cliente, al analista de riesgo, al gerente de aprobaciones, al departamento legal y al sistema de scoring. El Context Engineering debe considerar qué contexto es relevante para cada actor en cada etapa del proceso.

Menos estructura en los artefactos. Un proceso de atención al cliente opera sobre correos, conversaciones, tickets, historial de compras, políticas de devolución y conocimiento tácito del agente. No hay equivalente al `git log` que registre todas las decisiones del proceso con timestamp y autor.

Criterios de corrección más ambiguos. En código, "correcto" es verificable: el test pasa o no. En muchos procesos de negocio, "correcto" depende de juicios de valor, de contexto organizacional y de interpretación de políticas que no están completamente formalizadas.

Estas diferencias no invalidan los principios del capítulo 11 — los extienden. El mismo principio de contexto mínimo suficiente aplica; la misma distinción entre "el modelo amplifica, el humano decide" aplica; la misma importancia de verificar antes de integrar aplica. Lo que cambia es el mapa del territorio donde esos principios se aplican.

El puente conceptual

La relación entre el capítulo 11 y el capítulo 12 es la de lo particular a lo general. El Context Engineering para desarrollo de software es un caso especial del Context Engineering para procesos empresariales. Todos los patrones estudiados en el capítulo 11 son instancias de patrones más generales que el capítulo 12 analizará en su forma más amplia.

CAPÍTULO 11 → CAPÍTULO 12

Ciclo de vida del software	→ Procesos de negocio empresariales
Código, commits, tests	→ Correos, contratos, políticas, datos
Desarrolladores, arquitectos	→ Analistas, gerentes, clientes
Corrección verificable	→ Criterios de calidad organizacionales
Pipeline de CI/CD	→ Flujos de aprobación y decisión
Repositorio como fuente	→ Sistemas empresariales como fuente
Archivo de instrucciones	→ Políticas y contexto organizacional

Lo que el lector va a encontrar en el capítulo 12

El capítulo 12 aplica el Context Engineering a dominios como: atención al cliente, análisis de documentos legales, procesamiento de contratos, soporte a decisiones de negocio y automatización de flujos de trabajo transaccionales. En cada dominio, el análisis sigue la misma estructura que el capítulo 11: qué contexto existe en ese dominio, cómo seleccionarlo para tareas específicas, qué patrones funcionan y cuáles fallan.

El lector que completó el capítulo 11 tiene ya los principios operativos. El capítulo 12 aplica esos principios a nuevos territorios, ampliando el repertorio del AI Engineer más allá del dominio técnico hacia los procesos de negocio donde el impacto de los sistemas de IA es más amplio y, frecuentemente, más difícil de contener cuando algo falla.

Una nota final sobre el dominio del software

El hecho de que el capítulo 11 sea sobre desarrollo de software no es solo una conveniencia pedagógica. El AI Engineer que construye sistemas de IA para otros dominios también es un desarrollador de software. La comprensión del Context Engineering en su propio trabajo — cómo el contexto determina la calidad del código que él mismo genera con asistencia de IA — es una ventaja práctica inmediata, independientemente del dominio empresarial donde trabaje.

Lo que el lector aprendió sobre cómo construir el contexto para generar una función de Python correcta es aplicable mañana en su entorno de desarrollo. Lo que aprenderá en el capítulo 12 sobre procesos empresariales lo será cuando diseñe los sistemas de IA que esos procesos requieren.

El Módulo 3 continúa con el capítulo 12: Context Engineering para Procesos Empresariales.